

DATADOG

Log Management Fundamentals

Comprehensive Study Guide

2-Week Plan • Jun 1 → Jun 14, 2026

7 domains • 200+ hard practice questions • 2 mock exams

Prepared for

Marcio Costa

Sr Partner Solution Architect • Datadog Brazil

Contents

- About this guide
- Exam overview — format, content outline, and scoring
- 2-week study plan (June 1 → June 14, 2026)
- Domain 1 — Logging Fundamentals (deep dive + 22 quiz Q + answers)
- Domain 2 — Log Collection (deep dive + 22 quiz Q + answers)
- Domain 3 — Log Parsing (deep dive + 30 quiz Q + answers)
- Domain 4 — Log Searching & Filtering (deep dive + 28 quiz Q + answers)
- Domain 5 — Log Analysis (deep dive + 22 quiz Q + answers)
- Domain 6 — Log Utilization (deep dive + 22 quiz Q + answers)
- Domain 7 — Log Troubleshooting (deep dive + 18 quiz Q + answers)
- Mock Exam 1 — mid-prep (25 Q, mixed domains)
- Mock Exam 2 — final dress rehearsal (25 Q, harder scenarios)
- Quick reference cheat sheet
- Exam-day strategy & final tips

About this guide

This guide is a single-source, exam-focused study companion for the Datadog Log Management Fundamentals certification (January 2026 exam guide). It compresses the official exam content outline, the Datadog documentation, and battle-tested practice questions into a 2-week plan you can start on June 1st and finish ready to sit the exam by June 14th.

How to use it. Read each domain in order, take its quiz the same day, then revisit the explanations of any question you missed. The questions are harder than the real exam on purpose — if you can handle the trick distractors here, the actual exam will feel forgiving. Two full mock exams at the end simulate the cumulative format.

Format. The real exam is 90 questions (75 scored, 15 unscored pretest) in 120 minutes, multiple choice only, delivered by Kryterion Webassessor. Datadog does not publish a passing score; community reports suggest 80% as a safe target. Aim for ~85%+ on these mocks because they're calibrated harder than the live test.

Materials inside. Domain-by-domain content (concepts, configs, real query syntax, gotchas), 200+ hard quiz questions with full explanations and documentation references, a day-by-day study plan, a one-page cheat sheet, and exam-day notes.

Exam overview

Format at a glance

Attribute	Value
Total questions	90 (75 scored + 15 unscored pretest)
Question type	Multiple choice, single correct answer
Time limit	2 hours (120 minutes)
Delivery	Online proctored via Kryterion Webassessor
Cost	US\$100 per attempt
Credential validity	3 years
Retakes	Up to 3 attempts per 180-day window
Passing score	Not officially published; ~80% is the community-reported safe target

Content outline (from the January 2026 exam guide)

The Log Management Fundamentals exam validates that you can use Datadog Log Management end-to-end, independent of any specific job role. Datadog expects candidates to have at least six months of hands-on experience with the product.

The exam organizes content into seven knowledge domains, each broken down into subdomains. While Datadog does not publish exact percentage weights, the domain structure suggests the heaviest scoring areas are Log Parsing (processors and pipelines) and Log Searching & Filtering (Live Tail vs Explorer, search syntax). The lightest is typically Log Troubleshooting. Plan your time accordingly: 30-35 percent of your prep should go to parsing, 20-25 percent to search and filtering, and the remainder split across the other domains.

The seven knowledge domains and their subdomains:

Domain	Subdomains
1. Logging Fundamentals	Logging Systems · Log Sources · Logging Formats · Log Emission
2. Log Collection	Enabling Log Collection · Log Filtering & Obfuscation
3. Log Parsing	Processors & Pipelines · Standard Attributes · Log Composition
4. Log Searching & Filtering	Live vs Explorer · Search Syntax · Explorer Functionality
5. Log Analysis	Filtering and Excluding · Aggregations · Visualizing
6. Log Utilization	Monitors & Alerting · Metric Generation · Exporting
7. Log Troubleshooting	Agent Issues · Ingestion Issues

Out of scope

The exam does NOT test you on programming, systems architecture, or testing — these are listed as out-of-scope job tasks. Focus your prep on operating Datadog Log Management, not on building applications around it.

2-week study plan

Each day mixes reading, a hands-on lab in your Datadog org, and a targeted quiz from this guide. Total commitment: roughly 1.5 to 2 hours per day. Mock exams are scheduled on the two weekends.

Before you start. Have a Datadog account ready (free trial works). Install the Agent on a Linux VM or your laptop. Bookmark docs.datadoghq.com/logs/. Print the Quick reference cheat sheet at the end of this guide.

Week 1 — Foundations

Day 1 — Mon Jun 1 — Logging Fundamentals (Domain 1)

Morning (~45 min): Read Domain 1 fully. Trace each example through the log lifecycle diagram on paper. Spin up a Datadog free-trial org if you don't have one.

Evening (~30 min): Set up a Linux VM (or use an existing host) and install the Datadog Agent. Confirm ``logs_enabled: true`` is set and the Agent is running.

Quiz / checkpoint: Take the Domain 1 quiz (22 questions). Target: 80%+.

Day 2 — Tue Jun 2 — Log Collection (Domain 2)

Morning (~45 min): Read Domain 2 fully. Pay extra attention to `log_processing_rules` order and the four rule types.

Evening (~30 min): Tail a custom log file. Add an `exclude_at_match` rule to drop one specific line and confirm it doesn't appear in Live Tail.

Quiz / checkpoint: Take the Domain 2 quiz (22 questions). Target: 80%+.

Day 3 — Wed Jun 3 — Log Parsing — Part A (Domain 3, first half)

Morning (~45 min): Read Domain 3 sections 3.1 through 3.4 (pipelines, processors, Grok). Sketch the canonical processor order from memory.

Evening (~30 min): Create a custom log pipeline with a Grok parser against a real app log. Confirm the parsed attributes appear in the Explorer.

Quiz / checkpoint: No quiz today — consolidate.

Day 4 — Thu Jun 4 — Log Parsing — Part B (Domain 3, second half)

Morning (~45 min): Read Domain 3 sections 3.5 through 3.8 (reserved vs standard attributes, auto-JSON, log composition).

Evening (~30 min): Add a Date remapper and Status remapper to your pipeline. Trigger a known-bad log to confirm the remapper updates the official fields.

Quiz / checkpoint: Take the Domain 3 quiz (30 questions). Target: 80%+. This is the heaviest domain — review every miss.

Day 5 — Fri Jun 5 — Search & Filtering (Domain 4)

Morning (~45 min): Read Domain 4. Memorize the search syntax table — especially the ``@`` vs `no-`@`` rule and range operators.

Evening (~30 min): Run every query in section 4.2 against your live logs. Save 3 useful queries as Saved Views.

Quiz / checkpoint: Take the Domain 4 quiz (28 questions). Target: 80%+.

Day 6 — Sat Jun 6 — Catch-up + Lab day

Morning (~45 min): Review your Week 1 quiz misses. For each, re-read the explanation AND open the linked Datadog doc. Add the topic to a 'weak areas' note.

Evening (~30 min): Lab: set up a multi-line aggregation rule for a real stack trace. Confirm the stack appears as a single log in the Explorer.

Quiz / checkpoint: Optional: re-take any quiz where you scored below 75%.

Day 7 — Sun Jun 7 — Mid-prep Mock Exam 1 (25 Q)

Morning (~45 min): Rest morning. Read the cheat sheet at the end of this guide.

Evening (~30 min): Take Mock Exam 1 under exam conditions: 35 min, no notes, no docs. Score yourself; review every miss.

Quiz / checkpoint: Mock Exam 1 — record your score by domain in a 'progress' note.

Week 2 — Advanced

Day 8 — Mon Jun 8 — Log Analysis (Domain 5)

Morning (~45 min): Read Domain 5. Internalize the difference between search filter, index inclusion, index exclusion, and Agent edge filtering.

Evening (~30 min): Lab: create an index with an inclusion filter and a 50% exclusion filter on a noisy log. Confirm the sampling behavior in the Explorer.

Quiz / checkpoint: Take the Domain 5 quiz (22 questions). Target: 80%+.

Day 9 — Tue Jun 9 — Log Utilization — Monitors & Metrics (Domain 6 first half)

Morning (~45 min): Read Domain 6 sections 6.1 and 6.2 (Monitors, Generate Metrics).

Evening (~30 min): Lab: create a log monitor with a multi-alert on `service`. Create a logs-to-metrics rule with two group-by tags.

Quiz / checkpoint: No quiz today — consolidate.

Day 10 — Wed Jun 10 — Log Utilization — Archives & Forwarding (Domain 6 second half)

Morning (~45 min): Read Domain 6 sections 6.3 through 6.8 (Archives, Rehydration, Forwarding).

Evening (~30 min): Lab (optional, needs S3): configure an archive destination and confirm objects are written. Don't worry about rehydration — just understand the workflow.

Quiz / checkpoint: Take the Domain 6 quiz (22 questions). Target: 80%+.

Day 11 — Thu Jun 11 — Log Troubleshooting (Domain 7)

Morning (~45 min): Read Domain 7. Memorize the diagnostic funnel and the symptom → cause table.

Evening (~30 min): Lab: deliberately break something (stop the Agent, set wrong `source`, deny dd-agent read permission). Walk the funnel to diagnose.

Quiz / checkpoint: Take the Domain 7 quiz (18 questions). Target: 80%+.

Day 12 — Fri Jun 12 — Weak-area drill

Morning (~45 min): Open your 'weak areas' note. For each entry, re-read the corresponding domain section and the linked Datadog doc.

Evening (~30 min): Re-take the quiz of your two weakest domains. Aim for 90%+ on the retake (you've seen the questions once).

Quiz / checkpoint: Build a one-page personal cheat sheet of the 10 things you keep forgetting.

Day 13 — Sat Jun 13 — Mock Exam 2 — final dress rehearsal

Morning (~45 min): Rest morning. Skim the cheat sheet at the end of this guide. Read your personal one-pager.

Evening (~30 min): Take Mock Exam 2 under exam conditions: 40 min, no notes, no docs. Mock 2 is harder than mock 1 by design.

Quiz / checkpoint: Mock Exam 2 — your score should be 80%+ if you're ready. Below 75%? Spend Sunday reviewing your weakest domain hard.

Day 14 — Sun Jun 14 — Light review + exam day prep

Morning (~45 min): Light morning: re-read only your personal one-page cheat sheet and the official Quick Reference at the end of this guide. No new content.

Evening (~30 min): Schedule the exam if you haven't (Kryterion Webassessor). Sleep early. Have water and a snack ready.

Quiz / checkpoint: Day of exam: scan the cheat sheet once, sit early afternoon if possible, trust the prep.

Domain 1 — Logging Fundamentals

Logging Systems (rationale & justification) · Log Sources · Logging Formats · Log Emission

Domain 1 sets the conceptual baseline. The exam probes whether you understand WHY logs matter as one of the three pillars of observability, what kinds of systems and applications emit them, what formats Datadog can consume, and the patterns by which logs leave their source on the way to Datadog. Many exam answers hinge on the principle of Logging Without Limits: ingestion is decoupled from indexing, which means you can capture all logs cheaply and pay only for the subset you make searchable.

1.1 Why centralize logs in Datadog

Logs are immutable, timestamped records of events emitted by a system. In observability they answer WHY something happened, while metrics answer WHAT and traces answer WHERE. Their high-fidelity nature is also their cost driver: a modern service can produce terabytes of logs per day.

Organizations centralize logs in Datadog primarily for four reasons: troubleshooting (pivot from a metric or trace straight to the relevant log line), audit and compliance retention (PCI-DSS, HIPAA, SOC 2, GDPR), security analytics (Cloud SIEM detection rules feed off log streams), and business analytics (derive metrics like orders-per-minute from logs without modifying application code).

Pillar	Question it answers	Cardinality	Cost driver
Metrics	What is happening?	Low	Unique tag combinations
Traces	Where is it happening?	Medium	Indexed spans
Logs	Why is it happening?	High	Volume ingested + indexed

Logging Without Limits. The defining principle: ingest every log at a low rate; choose at index time which subset is searchable. Excluded logs still flow to Archives, generate metrics, and stream into Live Tail. Volume and cost are no longer the same lever.

1.2 Where logs come from (log sources)

The exam expects you to recognize the major source categories, not memorize integration lists.

- Application logs — your code writes to a file (most common on VMs) or to STDOUT/STDERR (12-factor / containers).
- Host / system logs — `/var/log/syslog`, `journald`, Windows Event Log channels (System / Application / Security), macOS unified logging.
- Container logs — the Agent reads from `/var/log/pods/` (Kubernetes) or from the Docker socket / per-container JSON files.
- Cloud-managed services — AWS via the Datadog Forwarder Lambda or Kinesis Firehose, Azure via Event Hub, GCP via Pub/Sub + Dataflow.
- Network appliances — syslog RFC 3164 / 5424 over TCP/UDP into a Datadog Agent listening on a port.
- Browser & mobile — the Browser SDK and Mobile RUM SDKs send directly to the HTTP intake.
- Custom apps that cannot run an Agent — direct POST to the HTTP Logs Intake API.

1.3 Log formats and how Datadog handles them

Datadog accepts everything from raw text to richly structured JSON, but the work you save downstream depends heavily on the format you emit.

Format	Example	Parsing effort
Unstructured	2026-05-03 14:22:01 INFO Order 4521 created	Grok parser required
logfmt (key=value)	ts=... level=info order_id=4521	Key/Value parser processor handles it
JSON	{"ts": "...", "level": "info", "order_id": 4521}	Auto-parsed; level & timestamp auto-mapped
Common Log / Combined Log	127.0.0.1 - frank [...] "GET /api" 200 2326	OOTB pipeline when source:nginx is set
Syslog (RFC 3164/5424)	<34>1 2026-05-03T14:22:01Z host app...	OOTB pipeline; Agent listens on TCP/UDP

Strongest exam recommendation. For any NEW application emit JSON. Datadog auto-parses every top-level key into an attribute, recognizes `level`/`severity` as `status`, recognizes `timestamp`/`date` variants as the log timestamp, and preserves numeric/boolean types.

1.4 How logs leave the source (emission patterns)

Method	When to use it
File tailing	Default for VMs and traditional apps; Agent tails files using globs.
STDOUT / STDERR	Containers; the runtime captures the stream; Agent reads via the runtime layer.
TCP / UDP socket	Network gear, appliances, syslog-based services that cannot write files.
HTTP Logs Intake API	Serverless, browser, edge — direct POST to <code>http-intake.logs.<site></code> .
AWS Lambda Forwarder / Extension	AWS-managed services (CloudTrail, VPC Flow, Lambda function logs).
Vector / Fluent Bit / Fluentd	Existing log pipelines you want to keep but also ship to Datadog.

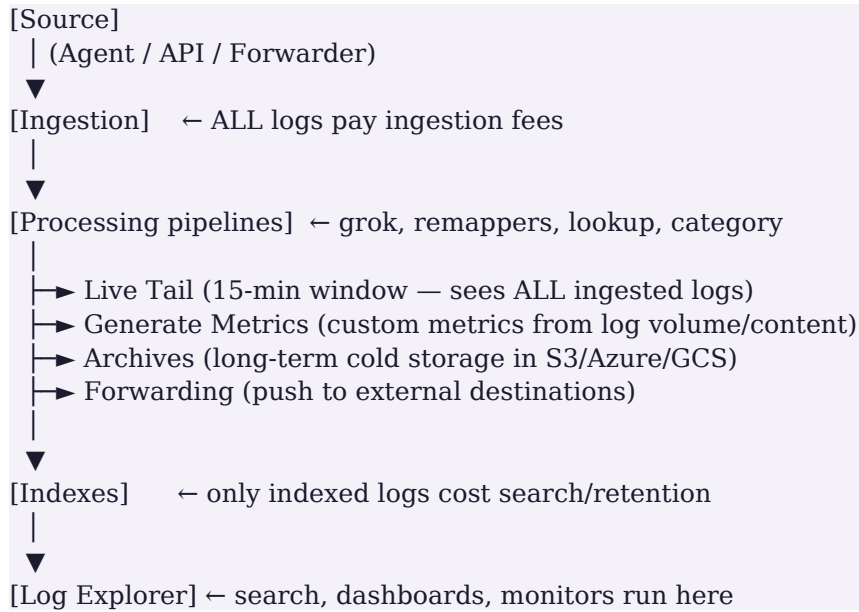
1.5 Severity levels

Datadog's reserved severity values, descending: emergency, alert, critical, error, warning, notice, info, debug, plus OK. Numeric aliases and abbreviations (`crit`, `err`, `warn`) are recognized by the Status remapper. A Status remapper sets the official reserved `status` attribute from any source field — usually `@level` or `@severity`. JSON `level` and `severity` keys are auto-recognized without a remapper.

1.6 The log lifecycle inside Datadog

Many exam questions hinge on this flow. Memorize the order.

- All ingested logs pass through processing pipelines, even those eventually excluded from indexing.
- Exclusion filters sit AFTER pipelines but BEFORE indexes — excluded logs still flow to archives and metric generation.
- Live Tail is a real-time view of everything ingested in the last ~15 minutes, regardless of indexing decisions.



1.7 Log-to-trace correlation (memorize)

For a log line to appear in the APM Trace side-panel, the log must carry ``dd.trace_id``, ``dd.span_id``, AND a matching ``service`` and ``env``. The Datadog tracer libraries inject these automatically when log injection is enabled (set ``DD_LOGS_INJECTION=true`` for most languages or use the language-specific helper).

1.8 Exam takeaways

- Logs answer WHY; metrics and traces answer what and where.
- Logging Without Limits decouples ingestion from indexing — the single most-tested concept.
- For new apps emit JSON. Datadog auto-parses keys and auto-maps level/severity/timestamp.
- All ingested logs flow through pipelines; exclusion filters live on indexes, after pipelines.
- Correlation requires ``dd.trace_id`` + ``dd.span_id`` + matching ``service`/`env``.

Domain 1 — Quiz (22 hard questions)

Answer all 22 first, then check the key on the next page. Target: 80%+. The real exam is easier.

Q1. A team has migrated their billing service from VMs to Kubernetes. After the migration, they emit logs to STDOUT as before, but they want logs to stay correlated with their existing APM traces (already running in the same env). Which set of fields **MUST** appear in each log line for the Trace side-panel to show them?

- A. `host`, `service`, `env`, and `pod\_name`
- B. `dd.trace\_id`, `dd.span\_id`, plus a matching `service` and `env`
- C. `@request\_id` and `@usr.id` (Datadog will auto-link by user)
- D. `trace.id` and `span.id` (OpenTelemetry standard) — Datadog auto-translates

Q2. Your platform team must persuade the CFO to centralize logs in Datadog. Which statement best captures the headline value of `Logging Without Limits`?

- A. Datadog stores all logs indefinitely at no extra cost.
- B. It removes the need for a dedicated SIEM by combining logs with metrics in a single index.
- C. It decouples log ingestion from log indexing, so volume and searchable-cost are no longer the same lever.
- D. It guarantees automatic parsing of every log format via machine learning.

Q3. A new microservice emits logs as JSON to STDOUT. Among these fields, which one will Datadog automatically promote to the official log severity **WITHOUT** requiring a Status remapper?

- A. `severity\_level`
- B. `@status`
- C. `level`
- D. `log.severity`

Q4. Which of the following is **NOT** one of the three pillars of observability that Datadog combines?

- A. Metrics
- B. Traces
- C. Logs
- D. Events

Q5. A legacy NGINX server writes the standard Combined Log Format to `/var/log/nginx/access.log`. What is the **SIMPLEST** way to get parsed `@http.method`, `@http.status\_code`, and `@http.url` attributes in Datadog?

- A. Write a custom Grok parser in a new pipeline for `source:nginx`.
- B. Set `source: nginx` in the Agent's log config — the OOTB integration pipeline does the parsing.
- C. Set `service: nginx` — Datadog uses the service name to match parsing rules.
- D. Forward logs through Fluent Bit with the `nginx\_parser` filter.

Q6. Which statement about reserved attributes vs. standard attributes is **TRUE**?

- A. Reserved attributes always start with `@`; standard attributes do not.
- B. Reserved attributes are the seven top-level fields (host, service, source, status, message, timestamp, tags); standard attributes are Datadog-curated `@\*` fields (e.g., `@http.\*`).

- C. Standard attributes can be edited; reserved attributes cannot.
- D. Both reserved and standard attributes require explicit faceting before they appear in the sidebar.

Q7. A team operating a low-latency trading platform asks: 'We need to keep 90 days of audit logs for SEC compliance, but we never search older than 30 days.' What is the most cost-effective architecture?

- A. Set the index retention to 90 days for all logs.
- B. Index for 30 days; send all logs to Archives (S3) with 90-day lifecycle; rehydrate when SEC requests arrive.
- C. Use Sensitive Data Scanner to redact then index 90 days.
- D. Enable Live Tail with extended retention to 90 days.

Q8. Which emission method is most appropriate for a Lambda function that runs for <30 seconds and is invoked thousands of times per minute?

- A. Install the Datadog Agent inside the Lambda runtime.
- B. Use the Datadog Lambda Extension (preferred) or the Datadog Forwarder Lambda.
- C. Write logs to `~/var/log/lambda.log` for the Agent to tail.
- D. Open a TCP socket from Lambda to Datadog intake.

Q9. Inside Datadog, what does the ``tags`` array on a log most commonly contain (assuming default Agent collection from a Linux host running an integration)?

- A. Only the tags configured on the integration's ``conf.yaml``.
- B. A merge of host tags, integration tags, environment tags, and any inline ``ddtags`` — with later sources overriding earlier ones.
- C. Tags extracted from the log's text via Grok.
- D. A single concatenated string of ``env:``, ``service:``, ``version:``.

Q10. Which of the following statuses is NOT a valid reserved log status in Datadog?

- A. emergency
- B. critical
- C. fatal
- D. notice

Q11. True or False: An excluded log (via an index exclusion filter at 100%) cannot be used to generate a custom metric.

- A. True
- B. False

Q12. A team wants to forward audit logs from their NGINX gateway to an on-prem syslog collector AND to Datadog. The cleanest architecture is:

- A. Configure NGINX to write to two log files; the Agent tails one, rsyslog reads the other.
- B. Use the Datadog Agent's built-in syslog-forwarder feature.
- C. Forward via Vector (or Fluent Bit) with two sinks: Datadog and syslog.

D. Have NGINX POST every line to both endpoints from Lua.

Q13. Which is the BEST description of the order of operations when a log arrives in Datadog?

- A. Index → Pipelines → Archive → Live Tail
- B. Ingestion → Pipelines → (Live Tail, Archives, Metrics) → Indexes → Explorer
- C. Pipelines → Ingestion → Indexes → Archives
- D. Agent → Index → Pipelines → Archives

Q14. A developer writes: `2026-05-03 14:22:01 INFO Order 4521 created` to a file. Which statement about this format is MOST accurate?

- A. Datadog will auto-parse this as if it were JSON because of the timestamp prefix.
- B. Datadog will require a Grok parser to extract fields; the `Status remapper` will not auto-fire on `INFO`.
- C. Auto-JSON parsing handles unstructured text as long as there's a timestamp.
- D. The Agent rejects unstructured logs unless `parser: text` is set.

Q15. For a brand-new Go application being written in 2026, what is the strongest log-emission recommendation?

- A. Emit logfmt to STDOUT — easiest for grep on the host.
- B. Emit structured JSON to STDOUT with `level`, `msg`, `timestamp`, plus Datadog trace IDs when applicable.
- C. Emit plain text to a file rotated nightly so the Agent can tail it.
- D. Emit XML for forward compatibility with enterprise SIEMs.

Q16. What is the function of the Datadog `Browser SDK` in relation to Log Management?

- A. It tails browser console logs from a Chrome extension.
- B. It sends front-end events (including console logs and custom events) directly to the Datadog HTTP intake.
- C. It captures HAR files and uploads them periodically.
- D. It replaces the Datadog Agent for SPAs.

Q17. Which is TRUE about Live Tail?

- A. It searches the last 24 hours of indexed logs.
- B. It shows all ingested logs in a streaming window of approximately the last 15 minutes, regardless of indexing.
- C. It requires logs to be in an index before they appear.
- D. It is the same as a Saved View with a 15-minute time range.

Q18. True or False: A custom JSON attribute is automatically searchable as `@attr:value` even if it is not declared as a facet.

- A. True
- B. False

Q19. Which of the following is the EARLIEST stage you can drop a log to reduce ingestion cost?

- A. Pipeline processor with a `Drop` action.
- B. Index exclusion filter at 100%.
- C. Agent `log\_processing\_rules` with `exclude\_at\_match`.
- D. Archive lifecycle rule on S3.

Q20. Which log emission pattern is described by the term '12-factor logging'?

- A. Write to 12 separate log files for parallel processing.
- B. Treat logs as event streams emitted to STDOUT/STDERR — collection is the platform's job.
- C. Always emit syslog over TCP for guaranteed delivery.
- D. Use 12 reserved attributes per log to maximize searchability.

Q21. A team's `@duration` is in nanoseconds but the Top List visualization shows raw numbers like `1500000000`. What is the cleanest fix so it reads as `1.5s`?

- A. Use a Grok `scale(0.000000001)` filter to convert nanos to seconds in the pipeline.
- B. Define `@duration` as a measure with unit `nanoseconds` so Datadog displays it formatted.
- C. Manually divide every value in a Notebook with a math expression.
- D. Switch to milliseconds by editing every application.

Q22. Which is the BEST troubleshooting first step when a user reports 'I added a `team:checkout` tag in the Agent config but it's missing from the logs in Datadog'?

- A. Open the pipeline editor and inspect each remapper.
- B. Restart the Agent — tag changes in `conf.yaml` require a restart to take effect.
- C. Re-create the facet so it picks up the new tag.
- D. Drop and recreate the index.

Domain 1 — Quiz (22 hard questions) — Answer Key & Explanations

Q1. Answer: B

Log-trace correlation in Datadog requires both ``dd.trace_id`` and ``dd.span_id`` in the log, AND the log's ``service`` and ``env`` must match the APM service. OpenTelemetry's ``trace.id`/`span.id`` are NOT auto-translated to Datadog's ``dd.`` namespace at log ingest. Enabling tracer log injection (e.g., ``DD_LOGS_INJECTION=true``) is the supported way to inject these.

Doc: docs.datadoghq.com/tracing/other_telemetry/connect_logs_and_traces/

Q2. Answer: C

``Logging Without Limits`` decouples ingestion from indexing: you ingest everything cheaply, then choose what to index for high-cost search. Excluded logs can still flow to archives, generate metrics, and be Live-Tailed. Datadog does NOT store everything for free, does NOT replace a SIEM by itself, and does NOT auto-parse every format (Grok parsers are still needed for unstructured text).

Doc: docs.datadoghq.com/logs/log_configuration/

Q3. Answer: C

Datadog auto-detects the JSON keys ``level`` and ``severity`` and uses them to set the official ``status``. Custom names like ``severity_level`` or ``log.severity`` require an explicit Status remapper. ``@status`` does not exist as a source field — ``status`` is the reserved attribute Datadog SETS, not a field you provide.

Doc: docs.datadoghq.com/logs/log_configuration/attributes_naming_convention/

Q4. Answer: D

The three pillars are Metrics (what), Traces (where), and Logs (why). Events are a Datadog feature but are not considered one of the canonical three pillars of observability.

Doc: docs.datadoghq.com/logs/

Q5. Answer: B

Out-of-the-box integration pipelines fire when the ``source`` tag matches a known integration (e.g., ``nginx``, ``apache``, ``kafka``). They are case-sensitive and triggered by ``source``, NOT by ``service``. Setting ``source: nginx`` is the simplest path; you only need a custom Grok when no OOTB pipeline exists.

Doc: docs.datadoghq.com/logs/log_configuration/pipelines/

Q6. Answer: B

Reserved attributes are the seven core top-level fields without ``@``: host, service, source, status, message, timestamp, tags. Standard attributes are Datadog's curated set of ``@*`` namespaces (e.g., ``@http.*``, ``@network.*``, ``@error.*``). Both are auto-faceted, so you do NOT have to create facets manually for them.

Doc: docs.datadoghq.com/logs/log_configuration/attributes_naming_convention/

Q7. Answer: B

Archives are the cheap long-term store (you pay your cloud storage rate, not Datadog index rate). Set index retention at the actual search horizon (30 days), and rely on archive + rehydration for the rare compliance pull. Live Tail does NOT have configurable retention (~15 min) and is not for compliance.

Doc: docs.datadoghq.com/logs/log_configuration/archives/

Q8. Answer: B

Lambda has no persistent host to install an Agent on, and the function's filesystem is ephemeral. Datadog offers the Lambda Extension (preferred, low-latency, runs as a layer) or the Forwarder (Lambda subscribed to CloudWatch). Direct TCP is impractical and would extend invocation time.

Doc: docs.datadoghq.com/serverless/installation/python/

Q9. Answer: B

Tags merge from multiple sources: Agent/host tags → integration `conf.yaml` tags → cloud/Kubernetes metadata tags → inline `ddtags` field on the log. Later sources override earlier ones on key collisions. Grok extracts attributes (`@\*`), not tags.

Doc: docs.datadoghq.com/getting_started/tagging/assigning_tags/

Q10. Answer: C

Datadog's reserved statuses are: emergency, alert, critical, error, warning, notice, info, debug, plus OK. `fatal` is NOT in the reserved list — many libraries emit it; the Status remapper must convert it (e.g., to `critical` or `error`).

Doc: docs.datadoghq.com/logs/log_configuration/processors/?tab=ui#log-status-remapper

Q11. Answer: B

FALSE. Logs-to-Metrics runs on ALL ingested logs, INDEPENDENT of indexing. Excluded logs still drive custom metrics and still flow to archives. The exclusion filter only removes them from the searchable Log Explorer index.

Doc: docs.datadoghq.com/logs/log_configuration/logs_to_metrics/

Q12. Answer: C

Vector and Fluent Bit are designed exactly for the fan-out pattern (one source → multiple sinks). The Agent does not natively act as a syslog forwarder; option A duplicates disk I/O; option D is fragile. Vector is Datadog's recommended log pipeline tool for complex routing.

Doc: docs.datadoghq.com/integrations/vector/

Q13. Answer: B

The lifecycle is: Ingestion → Pipelines (parse / enrich, always runs) → fan-out to Live Tail / Archives / Generate Metrics / Indexes (which are gated by inclusion + exclusion filters) → Explorer/dashboards/monitors query the indexes. Exclusion filters live AFTER pipelines but BEFORE indexing.

Doc: docs.datadoghq.com/logs/log_configuration/

Q14. Answer: B

Unstructured text is NOT auto-parsed. You need a Grok parser to extract `timestamp`, `level`, `order\_id`, etc. The Status remapper only fires when a `level` (or `severity`) attribute already exists — extract it first via Grok or by emitting JSON.

Doc: docs.datadoghq.com/logs/log_configuration/parsing/

Q15. Answer: B

JSON to STDOUT is the modern default: Datadog auto-parses every key into an attribute, auto-detects `level` and `timestamp`, keeps numeric types intact, and works identically in containers. logfmt parses fine but loses type info; XML is rarely supported; file-on-disk adds rotation complexity in containers.

Doc: docs.datadoghq.com/logs/log_collection/

Q16. Answer: B

The Browser SDK sends browser logs (console output, custom logger calls, errors) directly to Datadog's HTTP intake — no Agent involved. It complements RUM (Real User Monitoring) and lives entirely in the user's browser.

Doc: docs.datadoghq.com/logs/log_collection/javascript/

Q17. Answer: B

Live Tail streams logs as they are ingested in a ~15-minute rolling window — INCLUDING logs that are excluded from all indexes. It's the only place to verify that an excluded log is still being received.

Doc: docs.datadoghq.com/logs/explorer/live_tail/

Q18. Answer: A

TRUE. Free-text search by `@attribute:value` works on any indexed log even before you create a facet. Faceting only enables the LEFT SIDEBAR aggregation (top-N, ranges, etc.) and use as group-by in analytics. So you can find the log, but you can't aggregate on it until you facet it.

Doc: docs.datadoghq.com/logs/explorer/facets/

Q19. Answer: C

`exclude_at_match` in the Agent's `log_processing_rules` drops the log at the EDGE, before it leaves the host — so you pay no ingestion fee. Index exclusion filters drop after ingestion (still billed). Datadog pipelines have no `Drop` action. Archive lifecycle deletes already-stored data.

Doc: docs.datadoghq.com/agent/logs/advanced_log_collection/

Q20. Answer: B

12-factor (Heroku-popularized) treats logs as event streams written to STDOUT/STDERR. The app does not write to files; the platform (container runtime, log shipper, etc.) captures the stream. This is the default for containerized apps and what Kubernetes integrates with.

Doc: 12factor.net/logs

Q21. Answer: B

Defining `@duration` as a measure with the correct unit (nanoseconds) lets Datadog format values with human-readable units (`1.5 s`, `230 ms`) wherever the measure is displayed. Re-scaling at the pipeline destroys the original value; manual math doesn't persist; rewriting apps is overkill.

Doc: docs.datadoghq.com/logs/explorer/facets/#units

Q22. Answer: B

Tag changes in `datadog.yml` or integration `conf.yml` are NOT hot-reloaded — the Agent must be restarted. Pipelines, facets, and indexes all live in Datadog's backend and have no relationship to per-host tag changes that have not been picked up yet.

Doc: docs.datadoghq.com/agent/configuration/agent-configuration-files/

Domain 2 — Log Collection

Enabling Log Collection · Log Filtering & Obfuscation

Domain 2 tests whether you can configure the Datadog Agent to collect logs from a variety of sources and apply edge-level filtering, masking, and aggregation. Most failures here stem from forgetting the master switch (`logs_enabled: true`) or from misordering rules inside `log_processing_rules`.

2.1 The master switch

Log collection is OFF by default in the Datadog Agent. Turn it on in `datadog.yaml`:

Platform	Where <code>datadog.yaml</code> lives
Linux	<code>/etc/datadog-agent/datadog.yaml</code>
Windows	<code>C:\ProgramData\Datadog\datadog.yaml</code>
macOS	<code>~/datadog-agent/datadog.yaml</code>
Docker container	Use env var <code>DD_LOGS_ENABLED=true</code>
Kubernetes (Helm)	<code>datadog.logs.enabled: true</code> in <code>values.yaml</code>

```
logs_enabled: true
```

Trap. Without `logs_enabled: true` (or `DD_LOGS_ENABLED=true` as the env-var equivalent for containers), every other log setting in the Agent does nothing. This is THE most common exam trap.

2.2 Per-source configuration

Once logs are enabled globally, you tell the Agent WHAT to collect via files under `conf.d/<integration>.d/conf.yaml`. Each entry under `logs:` is a YAML list item. Mandatory fields: type, path (or port), service, source. Optional: tags, `log_processing_rules`, encoding, `exclude_paths`.

- type can be: file, tcp, udp, journald, windows_event, docker.
- Source triggers OOTB integration pipelines (case-sensitive — `nginx`, `apache`, `kafka`).
- Service ties logs to APM/RUM via Unified Service Tagging.
- Glob wildcards in path work (`/var/log/*.log`); recursive `***` does not — use multiple entries.
- The same `conf.yaml` can define multiple `logs:` entries.

```
logs:
- type: file
  path: "/var/log/myapp/app.log"
  service: "checkout-service"
  source: "java"
  tags:
  - env:prod
  - team:checkout
  log_processing_rules:
  - type: exclude_at_match
    name: drop_health
    pattern: "GET\\s+/health"
```

2.3 Container & Kubernetes collection

In containers, you almost always enable log collection via environment variables. The Agent reads container logs via the runtime's log files (under `/var/log/pods/`` on Kubernetes) — NOT from inside the container.

- ``container_collect_all: true`` collects STDOUT/STDERR from every container — NOT files inside containers.
- Per-pod customization uses Autodiscovery annotations with the exact container name from the pod spec.
- Datadog ships default integration mappings so common images (nginx, redis, mongo) get the right ``source`` automatically.

```
# Docker host
DD_LOGS_ENABLED=true
DD_LOGS_CONFIG_CONTAINER_COLLECT_ALL=true

# Kubernetes pod annotation (per-workload customization)
metadata:
  annotations:
    ad.datadoghq.com/<container-name>.logs: |
      [{
        "source": "java",
        "service": "checkout",
        "log_processing_rules": [
          { "type": "multi_line",
            "name": "log_start_with_date",
            "pattern": "\\d{4}-\\d{2}-\\d{2}" }
        ]
      }]
    }
```

2.4 Cloud-managed sources (no Agent)

- HTTP intake endpoint: `https://http-intake.logs.<site>/api/v2/logs` (NOT `api.<site>` — common distractor).
- Headers: ``DD-API-KEY: <key>``, ``Content-Type: application/json``.
- Limits: 5 MB per payload, 1 MB per individual log line.

Source	Collection path
AWS CloudWatch Logs	Datadog Forwarder Lambda subscribed to log groups
AWS CloudTrail / VPC Flow / S3 access	Forwarder reads from S3 bucket
AWS Lambda function logs	Datadog Lambda Extension (preferred) or Forwarder
Azure	Activity Logs / Event Hub / Diagnostic Settings + Azure integration
GCP	Pub/Sub + Datadog Dataflow template
Browser apps	Browser SDK → HTTP intake
Custom apps	Direct POST to <code>http-intake.logs.<site>/api/v2/logs</code>

2.5 Edge filtering & obfuscation — `log_processing_rules`

Four rule types in ``log_processing_rules``, executed IN ORDER within a single source. This is the only stage where you can drop logs BEFORE they leave the host (saving ingestion cost).

type	What it does
<code>exclude_at_match</code>	Drop log if regex matches
<code>include_at_match</code>	Keep ONLY logs that match
<code>mask_sequences</code>	Replace matched substring with a placeholder (PII)

	redaction)
multi_line	Lines NOT matching the pattern get appended to the previous matching line (stack-trace aggregation)

```
log_processing_rules:
- type: exclude_at_match
  name: drop_healthchecks
  pattern: GET\s+/\health
- type: mask_sequences
  name: mask_cc
  pattern: '\d{4}-\d{4}-\d{4}-\d{4}'
  replace_placeholder: "[CC]"
- type: multi_line
  name: stack_trace
  pattern: '\d{4}-\d{2}-\d{2}'
```

Order matters. Drop noise first (`exclude\_at\_match`) → then mask survivors (`mask\_sequences`) → then aggregate multi-line. Reordering wastes CPU and can produce incorrect output.

2.6 Sensitive Data Scanner (server-side scrubbing)

When edge masking isn't feasible (e.g., logs come via Lambda Forwarder or HTTP intake), use Sensitive Data Scanner from the Datadog UI. It runs AFTER ingestion but BEFORE indexing and archive write, so redactions persist in both. It supports library rules (credit cards, SSNs, AWS keys, JWT, IPv4/v6, email, phone numbers in many regions), custom regex, scanning groups (scoped to service/env), and actions: redact, hash, partial redact, or tag-only.

Compliance edge vs server. If your SOC requires data never to leave the host in plaintext, use the Agent's `mask\_sequences`. If you can tolerate server-side scrubbing (and need redaction in archives too), use Sensitive Data Scanner.

2.7 Tagging logs

Tags merge from multiple sources, with later overriding earlier on key collisions: (1) Agent host tags from `datadog.yaml`, (2) tags from the integration's `conf.yaml`, (3) cloud/Kubernetes metadata tags, (4) inline `ddtags` field on the log (HTTP intake).

Three reserved (a.k.a. Unified Service) tags you MUST set: `env`, `service`, `version`. They link logs to APM, RUM, infrastructure, and dashboards.

Cardinality warning. Tags become facets. Avoid extremely high-cardinality values as tags — raw user IDs, full URLs with query strings — keep those as attributes (`@user.id`, `@http.url`).

2.8 Common gotchas

- `logs\_enabled: true` (or the env-var equivalent) is required — without it everything else is ignored.
- The `dd-agent` Linux user needs read on the log file AND execute (traverse) on each parent directory. Use ACLs: `setfacl -m u:dd-agent:rx`.

- Glob paths work; recursive `\*\*` globs do NOT — use multiple entries.
- Docker `container\_collect\_all: true` reads STDOUT/STDERR via the runtime, not files inside containers.
- HTTP intake endpoint is `http-intake.logs.<site>`, NOT `api.<site>` (very common distractor).

Domain 2 — Quiz (22 hard questions)

Focus on rule order in `log_processing_rules` and the difference between Agent edge filtering and server-side scrubbing.

Q1. A Kubernetes Agent DaemonSet is running but ``kubectl exec datadog-agent-xxx -- agent status`` shows zero logs collected. The Helm values include ``datadog.logs.enabled: true``. What is the MOST LIKELY missing piece?

- A. ``datadog.logs.containerCollectAll: true`` is not set, so the Agent is not picking up container STDOUT/STDERR.
- B. The Agent is on the wrong site.
- C. Pod annotations are missing on every workload.
- D. The Agent needs an additional ``DD_KUBERNETES_LOGS=true`` env var.

Q2. Which ``log_processing_rules`` type would you use to keep only ERROR and FATAL lines from a verbose debug log on the Agent edge?

- A. ``exclude_at_match`` with pattern ``.*ERROR.*|.*FATAL.*``
- B. ``include_at_match`` with pattern ``(ERROR|FATAL)``
- C. ``mask_sequences`` with the appropriate regex
- D. ``multi_line`` with the level prefix

Q3. A SOC team mandates that credit card numbers must never leave the host in plaintext. Where should you mask them?

- A. Sensitive Data Scanner in the Datadog UI (server-side).
- B. An ``exclude_at_match`` rule in the Agent.
- C. A ``mask_sequences`` rule in ``log_processing_rules`` on the Agent.
- D. A Grok parser in a custom pipeline.

Q4. An app emits stack traces across many lines. Which Agent rule type and pattern strategy is correct?

- A. ``multi_line`` with a pattern matching the END of a stack trace, e.g., ``\s+at\s+``.
- B. ``multi_line`` with a pattern matching the START of a NEW log line, e.g., ``\d{4}-\d{2}-\d{2}``.
- C. ``include_at_match`` with a stack-trace regex.
- D. ``exclude_at_match`` to remove the stack lines.

Q5. True or False: ``log_processing_rules`` defined inside a single ``logs:`` entry execute in the order listed.

- A. True
- B. False

Q6. You enable container log collection. Where does the Agent read container logs from in a default Linux Kubernetes setup?

- A. From ``stdout`` of the Agent container via ``tail -f``.
- B. From files under ``/var/log/pods/`` (mounted into the Agent).
- C. By exec-ing into each container.

D. Via the Kubernetes API ``pods/<name>/log`` endpoint.

Q7. Which is the correct order of tag merging on a log (later sources override earlier)?

- A. Inline ``ddtags`` → integration ``conf.yaml`` → host tags → cloud metadata
- B. Host tags → integration ``conf.yaml`` → cloud/K8s metadata → inline ``ddtags``
- C. Cloud metadata → host tags → integration conf → inline ``ddtags``
- D. Integration conf → cloud metadata → inline ``ddtags`` → host tags

Q8. A service writes to ``/var/log/foo/*.log`` but only ``/var/log/foo/app.log`` is being collected. The Agent's status says it's tailing one file. The most likely cause is:

- A. Recursive glob ``***`` not supported — but ``*.log`` IS — so the cause is elsewhere.
- B. The Agent has not been restarted since the new files were created.
- C. Only one entry per ``logs:`` block is allowed.
- D. Glob patterns in Datadog match only the most recently modified file.

Q9. An app sends logs directly to ``https://http-intake.logs.datadoghq.com/api/v2/logs``. Single payload size is rejected as too large. The hard limit is:

- A. 1 MB per payload
- B. 5 MB per payload
- C. 10 MB per payload
- D. 256 KB per payload

Q10. Which env var on the Agent container is the canonical way to set the master switch for log collection?

- A. ``DD_LOGS=true``
- B. ``DD_LOGS_ENABLED=true``
- C. ``DD_LOG_COLLECTION_ON=true``
- D. ``DD_AGENT_LOGS=enabled``

Q11. Which feature is NOT a function of ``log_processing_rules``?

- A. Exclude log lines matching a regex.
- B. Aggregate multi-line stack traces.
- C. Mask credit card numbers before transmission.
- D. Set the ``source`` attribute on the log.

Q12. You added a ``team:billing`` tag in the Agent config but it doesn't appear on logs. After confirming the Agent was restarted, what's the next thing to check?

- A. Whether a pipeline processor is overwriting the tag.
- B. Whether the YAML indentation is correct — ``tags:`` must be inside the ``logs:`` entry, not at the file root.
- C. Whether the tag was added to a facet.
- D. Whether the Sensitive Data Scanner is redacting tags.

Q13. Sensitive Data Scanner runs:

- A. On the Agent before transmission.
- B. Server-side, after ingestion, before indexing — it CAN redact data in archives if configured.
- C. Only at archive write time.
- D. Inside the customer's VPC via a managed connector.

Q14. Which is the BEST way to customize log parsing for a specific Kubernetes Deployment without editing each Agent config?

- A. Add ``ad.datadoghq.com/<container-name>.logs`` annotations on the Pod template.
- B. Edit ``datadog.yaml`` on every node.
- C. Use a ConfigMap mounted into each Pod.
- D. Set environment variables on the Agent DaemonSet.

Q15. True or False: Setting ``container_collect_all: true`` on the Agent collects logs from files INSIDE each container, not from STDOUT/STDERR.

- A. True
- B. False

Q16. Which one of these is the supported method to read Windows Event Log via the Datadog Agent?

- A. ``type: file`` with path ``C:\Windows\System32\winevt\Logs\*.evtx``
- B. ``type: windows_event`` with ``channel_path: "System"`` or similar
- C. Custom PowerShell scheduled task forwarding to a TCP socket on the Agent
- D. ``type: journald`` (the Windows compatibility layer)

Q17. A regulator demands customer SSN redaction in archives, not just in indexes. Which configuration achieves this?

- A. Sensitive Data Scanner with the SSN rule applied — it scrubs before both indexing AND archive write.
- B. An index exclusion filter on logs containing SSNs.
- C. A Grok parser that doesn't extract the SSN.
- D. Agent ``mask_sequences`` is NOT supported for SSNs.

Q18. Which combination of permissions does the ``dd-agent`` Linux user need to tail ``/var/log/app/app.log``?

- A. Read on the file only.
- B. Read on the file AND execute (traverse) on the parent directory.
- C. Read and write on the file.
- D. Membership in the ``root`` group.

Q19. Which ``type`` is correct for collecting logs from a network appliance that emits syslog over UDP port 514?

- A. ``type: file``
- B. ``type: udp`` with ``port: 514``
- C. ``type: syslog`` with ``protocol: udp``

D. ``type: network`` with ``port: 514``

Q20. A team writes ``service: web`` in their Agent log config for a Python Flask app. Later, in the Pipelines view, a Service remapper is configured with ``service`` from ``@app.name``. Which value wins for a given log?

- A. The Agent's ``service: web`` (collection time wins).
- B. The pipeline's Service remapper (downstream processing overrides).
- C. Whichever is alphabetically first.
- D. The Agent loses only if the log carries ``@dd.service``.

Q21. What is ``DD_CONTAINER_INCLUDE_LOGS`` used for?

- A. Enables Live Tail for containers.
- B. Allow-lists which containers' logs are collected by ``container_collect_all``.
- C. Forces every container to write logs to a shared volume.
- D. Enables JSON parsing for container logs.

Q22. Which is FALSE about Datadog Sites?

- A. ``datadoghq.com`` is the US1 site.
- B. ``datadoghq.eu`` is the EU1 site.
- C. An Agent configured for ``site: datadoghq.eu`` can still send to a US1 org if the API key matches.
- D. Misconfigured ``site:`` causes logs to silently go to the wrong region's intake.

Domain 2 — Quiz (22 hard questions) — Answer Key & Explanations

Q1. Answer: A

`logs.enabled` is the master switch, but in containers you must also opt-in to collecting from every container — `datadog.logs.containerCollectAll: true` (Helm) or `DD\_LOGS\_CONFIG\_CONTAINER\_COLLECT\_ALL=true`. Otherwise the Agent only reads logs for containers that explicitly declare their config via labels/annotations.

Doc: docs.datadoghq.com/containers/kubernetes/log/

Q2. Answer: B

`include\_at\_match` keeps only matching lines — exactly what you want when filtering noisy debug down to errors. `exclude\_at\_match` does the opposite (drops matches). `mask\_sequences` redacts, doesn't filter. `multi\_line` aggregates lines into one log.

Doc: docs.datadoghq.com/agent/logs/advanced_log_collection/

Q3. Answer: C

Only the Agent runs ON THE HOST, before the log is transmitted. `mask\_sequences` redacts in-place at the edge. Sensitive Data Scanner runs server-side AFTER ingestion — the plaintext has already left the host (a SOC failure for strict compliance). Grok extracts, doesn't redact.

Doc: docs.datadoghq.com/agent/logs/advanced_log_collection/?tab=configurationfile#scrub-sensitive-data-from-your-logs

Q4. Answer: B

`multi\_line` works by: lines that do NOT match the pattern are appended to the previous matching line. So the pattern must match the START of a new log line (timestamp, log-level prefix). Matching the END would merge the wrong lines. Stack traces typically lack a leading timestamp, so they get appended to the originating log line — exactly what you want.

Doc: docs.datadoghq.com/agent/logs/advanced_log_collection/?tab=configurationfile#multi-line-aggregation

Q5. Answer: A

TRUE. Order matters: typically drop noise (`exclude\_at\_match`) first, then mask survivors (`mask\_sequences`), then aggregate multi-line. If you swap order you can waste CPU masking lines you were going to drop anyway.

Doc: docs.datadoghq.com/agent/logs/advanced_log_collection/

Q6. Answer: B

Modern container runtimes (containerd, CRI-O, Docker with `json-file` driver) write per-container log files under `/var/log/pods/` (and `/var/log/containers/` symlinks). The Agent DaemonSet mounts that path and tails the files. Hitting `pods/<name>/log` via the API would not scale and is not how the integration works.

Doc: docs.datadoghq.com/containers/kubernetes/log/

Q7. Answer: B

The merge order (earlier → overridden by → later) is: host tags from `datadog.yaml` → integration `conf.yaml` `tags:` → environment/cloud/K8s auto-tags → inline `ddtags` on the log payload (for HTTP intake). On collisions, later overrides earlier.

Doc: docs.datadoghq.com/getting_started/tagging/assigning_tags/

Q8. Answer: A

Single-level globs (`\*.log`) work fine. The likely cause is that other files don't yet exist or don't have read permission for `dd-agent`. The other options are myths: the Agent watches the directory and picks up new files, `logs:` blocks accept multiple entries, and there's no 'most recent file' rule.

Doc: docs.datadoghq.com/agent/logs/?tab=tailfiles#custom-log-collection

Q9. Answer: B

Hard limits on the HTTP intake: 5 MB per request payload, with individual log lines capped at 1 MB. Beyond that the API rejects. Batches should be sized below 5 MB.

Doc: docs.datadoghq.com/api/latest/logs/#send-logs

Q10. Answer: B

``DD_LOGS_ENABLED=true`` is the env-var equivalent of ``logs_enabled: true`` in ``datadog.yaml``. The others are not valid Datadog env vars.

Doc: docs.datadoghq.com/agent/logs/

Q11. Answer: D

``source``, ``service``, and ``tags`` are set as siblings of ``log_processing_rules``, not inside it. ``log_processing_rules`` is restricted to four types: ``exclude_at_match``, ``include_at_match``, ``mask_sequences``, ``multi_line``.

Doc: docs.datadoghq.com/agent/logs/advanced_log_collection/

Q12. Answer: B

A common mistake: ``tags:`` indented at the wrong level so the parser treats it as a separate root key (or ignores it). It must be nested inside the specific ``logs:`` entry, OR set at the global host-tag level in ``datadog.yaml``. Sensitive Data Scanner doesn't touch tags; faceting doesn't affect tag presence; pipelines normally don't strip tags unless explicitly configured.

Doc: docs.datadoghq.com/agent/logs/log_collection/

Q13. Answer: B

Sensitive Data Scanner is a managed server-side feature that runs after ingestion and before indexing/archive. Configured rules can redact, hash, or partially redact. It does NOT run on the Agent; for true edge masking use ``mask_sequences``.

Doc: docs.datadoghq.com/sensitive_data_scanner/

Q14. Answer: A

Pod-level annotations using the Autodiscovery prefix ``ad.datadoghq.com/<container-name>.logs`` is the supported, scalable, declarative way to customize per-workload log handling (source, service, multi-line rules) without touching node-level Agent config.

Doc: docs.datadoghq.com/containers/kubernetes/log/?tab=helm#autodiscovery

Q15. Answer: B

FALSE. ``container_collect_all`` collects from each container's STDOUT/STDERR (captured by the container runtime as files under ``/var/log/pods/...``). Files inside a container's overlay filesystem are NOT accessible — that's why 12-factor logging to STDOUT is the convention.

Doc: docs.datadoghq.com/containers/kubernetes/log/

Q16. Answer: B

Datadog's Agent ships a dedicated ``windows_event`` log type that reads via the Windows Event Log API (channel paths: ``System``, ``Application``, ``Security``, custom). Reading raw ``evtx`` files would miss live events and rotation handling.

Doc: docs.datadoghq.com/agent/logs/?tab=tailfiles#windows-event-logs

Q17. Answer: A

Sensitive Data Scanner runs server-side AFTER ingestion but BEFORE indexing AND archive write, so redactions persist in both. Exclusion filters only remove from the searchable index, not from archives. Grok extracts; it doesn't redact what's already in ``message``.

Doc: docs.datadoghq.com/sensitive_data_scanner/

Q18. Answer: B

POSIX requires execute (search) permission on every directory in the path to a file. So `dd-agent` needs `r` on the file AND `x` on the parent directories. Use ACLs (`setfacl -m u:dd-agent:rx`) to grant precisely this without changing global ownership.

Doc: docs.datadoghq.com/agent/troubleshooting/permissions/

Q19. Answer: B

Datadog's Agent log types are: `file`, `tcp`, `udp`, `journald`, `windows_event`, `docker`. Syslog is just text over TCP/UDP, so you use `type: udp` (or `tcp`) with the port. There is no `type: syslog` or `type: network`.

Doc: docs.datadoghq.com/agent/logs/?tab=tcpudp

Q20. Answer: B

Pipeline processors run AFTER ingestion and can overwrite reserved attributes. A Service remapper explicitly designed to set `service` from `@app.name` will overwrite the Agent's value. Use the remapper deliberately, or remove it if you want collection-time tagging to be canonical.

Doc: docs.datadoghq.com/logs/log_configuration/processors/?tab=ui#service-remapper

Q21. Answer: B

`DD_CONTAINER_INCLUDE_LOGS` (and its sibling `DD_CONTAINER_EXCLUDE_LOGS`) are filter env vars that allow- or deny-list containers BY tag/name/image when `container_collect_all` is on. Useful when you want everything except a few noisy sidecars.

Doc: docs.datadoghq.com/agent/configuration/agent-configuration-files/#container-discovery-management

Q22. Answer: C

An Agent only sends to the site it is configured for. The API key must match the org in that region — keys are region-specific. The third option is a common myth: sites are isolated, and orgs do not exist across regions.

Doc: docs.datadoghq.com/getting_started/site/

Domain 3 — Log Parsing

Processors & Pipelines · Standard Attributes · Log Composition

Domain 3 is the heaviest section of the exam. Expect ordering questions (which processor runs first?), 'which processor would you add to solve X?', and scenario questions about why a parsed log shows up with the wrong attribute set. Internalize the canonical processor order, the difference between reserved and standard attributes, and what auto-JSON parsing does for free.

3.1 The pipeline model

A pipeline is an ordered set of processors that runs against logs matching a FILTER QUERY. Pipelines are evaluated top to bottom in the Pipelines page. CRUCIALLY: a log passes through every pipeline whose filter matches, in order — pipelines do NOT short-circuit. This is the opposite of indexes (where the FIRST matching index wins).

- Integration pipelines are pre-built by Datadog and triggered by ``source:<name>``. They are read-only — clone to customize.
- Custom pipelines live alongside integration pipelines and can be reordered.
- One level of nested pipelines is supported (an outer pipeline acts as a filter gate; inner processors run only if the outer matched).

3.2 The processor toolbox

Processor	What it does
Grok parser	Extracts attributes from a string using named regex patterns
Date remapper	Sets the official log timestamp from an attribute
Status remapper	Sets the official <code>`status`</code> (severity) from an attribute
Service remapper	Sets the official <code>`service`</code> from an attribute
Message remapper	Sets the official <code>`message`</code> (the headline) from an attribute
Attribute remapper	Renames / moves / copies attributes
URL parser	Splits URLs into host, path, query parameters
User-Agent parser	Splits UA string into browser / OS / device
Category processor	Buckets logs into a named category by attribute value
Arithmetic processor	Computes a new numeric attribute from others
String builder	Concatenates attributes (with literals) into a new string
GeoIP parser	Resolves IP → country / city / coordinates
Lookup processor	Translates a value via an inline table (small)
Reference Table lookup	Translates via a managed CSV (large, shareable)
Trace ID remapper	Promotes a custom attribute to <code>`dd.trace_id`</code>

3.3 Canonical processor order

Inside a pipeline, processors run in declaration order. The recommended sequence is:

- 1. Grok parser — extract attributes from raw ``message``.
- 2. Date remapper — set timestamp from the extracted attribute.
- 3. Status remapper — set severity.
- 4. Service / Message remappers — set the remaining reserved fields.
- 5. Standard attribute remappers — align custom names to ``@http.*``, ``@network.*``, etc.

- 6. Enrichment — GeoIP, User-Agent, Category, Lookup, Arithmetic.
- 7. Restructure — Attribute remappers, String builder.

Most common bug. Date remapper runs BEFORE the Grok that produced the timestamp attribute. Result: log arrives with ingest-time timestamp because the Date remapper had nothing to consume. Always extract first, then remap.

3.4 The Grok parser in depth

Grok patterns are named regexes. Datadog's syntax: `%{<MATCHER>:<EXTRACT>[:<FILTER>]}`. Built-in matchers include ``notSpace``, ``data``, ``word``, ``integer``, ``number``, ``date(format)``, ``boolean``, ``ipv4``, ``ipv6``, ``uuid``, ``regex(...)``. Useful filters: ``lowercase``, ``uppercase``, ``nullIf("...")``, ``keyvalue``, ``json``, ``array(...)``, ``scale(N)``.

- Match rules are evaluated top to bottom; first match wins.
- Support rules are reusable named patterns referenced by match rules (think helper functions).
- Go's RE2 regex engine — no lookahead/lookbehind, no backreferences.

```
# Input log:
2026-05-03 14:22:01 INFO [user=42] Order 4521 created

# Grok match rule:
my_rule %{date("yyyy-MM-dd HH:mm:ss"):timestamp} %{word:level} \[user=%{integer:user_id}\]
Order %{integer:order_id} %{data:action}
```

3.5 Reserved vs standard attributes

Type	Examples	Prefix
Reserved	host, service, source, status, message, timestamp, tags	no `@`
Standard @http.*	@http.method, @http.status_code, @http.url, @http.useragent	@
Standard @network.*	@network.client.ip, @network.bytes_read, @network.client.geoip.country.iso_code	@
Standard @error.*	@error.kind, @error.message, @error.stack	@
Standard @usr.*	@usr.id, @usr.email, @usr.name	@
Standard @db.*	@db.statement, @db.system, @db.user	@
Standard @duration	numeric, nanoseconds	@
Correlation	@trace_id, @span_id (also dd.trace_id, dd.span_id)	@ / dd.

Why this matters. OOTB dashboards rely on the standard attribute names. If your app emits ``request_status_code``, the HTTP dashboard reports zero traffic. Add a Standard Attribute remapper to align.

3.6 Auto-JSON parsing — what you get for free

If a log arrives as valid JSON, Datadog automatically:

- Promotes every top-level key to a queryable attribute (no Grok needed).
- Detects and uses these JSON keys AS reserved attributes: `host`, `service`, `source`, `status`, `message`, `timestamp`.
- Maps `level` and `severity` JSON keys to the reserved `status` (auto Status remapper).
- Maps recognized timestamp formats (RFC 3339, ISO 8601, common epoch variants) to the official `timestamp` (auto Date remapper).

Why JSON is the recommended format. Most pipeline work vanishes. You only build pipelines for enrichment, standardization, and the cases auto-detection doesn't cover.

3.7 Log composition (the envelope)

- Query tags / reserved attributes WITHOUT `@`: `service:checkout`, `env:prod`, `status:error`.
- Query attributes WITH `@`: `@http.status\_code:500`, `@duration:>1000000000`.

```
{
  "host": "web-01",
  "service": "checkout",
  "source": "java",
  "status": "error",
  "message": "Database timeout after 5000ms",
  "timestamp": "2026-05-03T14:22:01.000Z",
  "tags": ["env:prod", "team:checkout", "version:1.4.2"],
  "@http.method": "POST",
  "@http.url": "/api/v1/orders",
  "@http.status_code": 500,
  "@error.kind": "TimeoutException",
  "@duration": 5012000000,
  "dd.trace_id": "5894812345..."
}
```

3.8 Exam takeaways

- Pipelines run top to bottom; logs go through EVERY matching pipeline.
- Inside a pipeline, processors run in declaration order — reorder fixes most parsing bugs.
- Grok parser, Date remapper, Status remapper are the three you must know cold.
- Reserved attributes = no `@`. Standard attributes = `@\*` with Datadog-blessed names.
- JSON logs are auto-parsed; `level`/`severity` and `timestamp` are auto-remapped.
- Integration pipelines trigger on EXACT `source:<name>` match (case-sensitive).

Domain 3 — Quiz (30 hard questions)

The heaviest domain on the exam. Memorize canonical processor order and the reserved-vs-standard attribute distinction.

Q1. A custom pipeline contains, in order: Grok parser → Date remapper (on `@timestamp`) → Status remapper (on `@level`). Logs arrive with the correct severity but the wrong timestamp. The MOST LIKELY cause is:

- A. The Grok parser is failing to extract `@timestamp`, so the Date remapper has nothing to consume.
- B. The Date remapper must come BEFORE the Grok parser.
- C. The Status remapper is overwriting the timestamp.
- D. The pipeline filter is not matching the log.

Q2. Which Grok statement is INVALID Datadog syntax?

- A. `%{date("yyyy-MM-dd HH:mm:ss"):timestamp}`
- B. `%{integer:user_id}`
- C. `%{notSpace:level:lowercase}`
- D. `{integer:user_id}`

Q3. Two pipelines, A (filter `service:checkout`) and B (filter `source:java`), both match a log. Which is TRUE?

- A. Only pipeline A runs (the first matching pipeline wins).
- B. Both pipelines run, in the order they appear in the Pipelines page.
- C. The log is duplicated, one copy per pipeline.
- D. A merged virtual pipeline runs once.

Q4. The Grok filter `%{notSpace:url:nullIf("-")}` is most useful for:

- A. Trimming whitespace from extracted strings.
- B. Replacing the literal string `-`` (common 'missing value' marker in web logs) with a null attribute.
- C. Rejecting logs that contain `-`` in the URL.
- D. Converting `-`` to a Unix-style null character.

Q5. Which is NOT a processor type available in Datadog log pipelines?

- A. Grok parser
- B. User-Agent parser
- C. Regex Replace processor (used to mutate `message` after parsing)
- D. Arithmetic processor

Q6. A team needs to bucket `@duration` into `fast` (<100ms), `medium` (100-500ms), `slow` (>500ms). Best processor:

- A. Arithmetic processor
- B. Category processor
- C. Lookup processor
- D. String builder

Q7. Standard attribute remappers are most useful when:

- A. You need to deduplicate logs.
- B. Your application emits ``request_status_code`` but the OOTB HTTP dashboard expects ``@http.status_code``.
- C. You want to convert nanoseconds to milliseconds.
- D. You want to add IP geolocation.

Q8. Which is the BEST description of an integration pipeline?

- A. A user-editable pipeline that ships with Datadog and is triggered by ``service:<integration>``.
- B. A built-in, READ-ONLY pipeline triggered by ``source:<integration>``. To customize, clone it and edit the clone.
- C. An automatic translation layer that converts OTel logs to Datadog format.
- D. A pipeline that runs only when the integration is enabled in the Datadog Marketplace.

Q9. Two pipeline approaches are debated: (A) one giant pipeline with filter ``*``, (B) several pipelines each with a tight filter. Which is generally recommended?

- A. A — fewer pipelines is simpler.
- B. B — narrow filters reduce processing cost and make logic easier to reason about.
- C. Doesn't matter — Datadog pre-compiles them into one execution plan.
- D. A — Datadog only allows 10 pipelines per account.

Q10. An incoming JSON log has top-level ``severity: ERROR``. Which is TRUE?

- A. Datadog ignores it — only ``level`` is auto-detected.
- B. Datadog auto-promotes both ``level`` AND ``severity`` JSON keys to ``status`` without a Status remapper.
- C. A Status remapper on ``@severity`` is required.
- D. Datadog converts ``ERROR`` to ``5`` and stores the numeric form.

Q11. Inside a Grok parser, what does the ``Support rules`` section do?

- A. Sets fallback values for failed matches.
- B. Provides reusable named patterns that match rules can reference like helper functions.
- C. Defines the order in which match rules execute.
- D. Specifies error handling for the parser.

Q12. Within a single pipeline, processors run:

- A. In parallel for performance.
- B. In the declared order, top to bottom.
- C. In the optimal order auto-determined by Datadog.
- D. In alphabetical order by processor name.

Q13. What does the Trace ID remapper specifically do?

- A. Removes trace IDs from logs to reduce cardinality.

- B. Promotes an attribute (e.g., `@my_trace_id`) into `dd.trace_id` so the log-trace correlation works.
- C. Translates OpenTelemetry trace IDs to a shorter format.
- D. Replaces trace IDs with their span IDs.

Q14. A Grok pattern uses `%{number:latency_ms:scale(0.001)}`. The input is `latency_ms=1500`. What attribute value is stored?

- A. 1500
- B. 1.5
- C. 0.0015
- D. 1500000

Q15. A Lookup processor and a Reference Table both translate values. The most accurate distinction:

- A. Lookup is for static tables of < ~10k rows defined in the processor UI; Reference Table is for larger, version-controlled CSVs uploaded centrally.
- B. Lookup is for IP addresses; Reference Table is for user IDs.
- C. Lookup is server-side; Reference Table is Agent-side.
- D. They are identical — the names are synonyms.

Q16. Which is TRUE about nested pipelines?

- A. Datadog supports unlimited pipeline nesting.
- B. One level of nesting is supported; the inner pipeline only runs if the outer pipeline's filter matched.
- C. Nested pipelines run in reverse order (innermost first).
- D. Nesting is only supported for integration pipelines.

Q17. A log has top-level JSON field `host: web-01`. Will Datadog use it as the reserved `host` attribute?

- A. Yes — `host` is auto-detected as a reserved attribute.
- B. No — only the Agent sets `host`; JSON `host` is moved to `@host`.
- C. Yes, but only if the Agent's `host` is empty.
- D. No — `host` cannot be set via JSON.

Q18. Which of these processor outputs CAN be used as a measure (i.e., numeric aggregation)?

- A. Grok-extracted `@user_name` (string)
- B. Arithmetic processor output `@bytes_sent` (number)
- C. Category processor output `@latency_bucket` (string)
- D. Date remapper-set `timestamp`

Q19. An `@http.url` attribute looks like `/api/v1/orders/12345?token=abc`. To analyze traffic by route family without per-order cardinality, the BEST pattern is:

- A. Tag every log with the full URL.
- B. Use the URL parser to populate `@http.url_details.path`, then aggregate on `@http.url_details.path` (or further parse the path).

- C. Use a Status remapper.
- D. Drop all logs containing URLs.

Q20. What is the role of the Service remapper?

- A. Set the official `service` field from an attribute (e.g., `@app.name`).
- B. Map the `service` to a category like 'tier-1' or 'tier-2'.
- C. Replicate logs across services.
- D. Strip the `service` field for privacy.

Q21. Which is TRUE about the Message remapper?

- A. It sets the official `message` field from an attribute, replacing the raw log.
- B. It deletes the `message` to save storage.
- C. It is required for JSON logs.
- D. It only runs in integration pipelines.

Q22. A NGINX log line: `192.168.1.1 - - [03/May/2026:14:22:01 +0000] "GET /api HTTP/1.1" 200 532`. Which `@network.client.ip` value will the OOTB NGINX pipeline produce?

- A. The hostname of the NGINX server
- B. `192.168.1.1`
- C. Empty until GeoIP parser runs
- D. The downstream upstream IP

Q23. Which is the correct order of processors for the canonical 'extract → set timestamp → set severity → enrich' flow?

- A. Date remapper → Grok parser → Status remapper → GeoIP
- B. Grok parser → Date remapper → Status remapper → GeoIP
- C. Grok parser → Status remapper → Date remapper → GeoIP
- D. Status remapper → Grok parser → Date remapper → GeoIP

Q24. Grok pattern `%{notSpace:level:lowercase}` extracts and:

- A. Lowercases the captured value.
- B. Matches only lowercase tokens.
- C. Stores the original case but indexes it as lowercase.
- D. Throws an error if the source isn't already lowercase.

Q25. Which of these JSON keys would NOT be auto-promoted to the reserved attribute set?

- A. host
- B. service
- C. log_id
- D. timestamp

Q26. A log carries an attribute `@order.total` (number). To enable `avg(@order.total)` group-by, you must:

- A. Create a facet on `@order.total`.
- B. Create a MEASURE on `@order.total` (which is a facet with a unit).
- C. Convert it to a tag with `unit:currency`.
- D. Nothing — numeric attributes are always aggregatable.

Q27. Pipeline filter `service:(billing OR orders) env:prod`. Which log matches?

- A. A log with `service:billing env:prod`
- B. A log with `service:checkout env:prod`
- C. A log with `service:billing env:staging`
- D. A log with `service:orders env:dev`

Q28. Two messages need to be combined: `request_id=abc123` and `latency=540`. Which processor creates a single new attribute like `@request_summary`?

- A. Arithmetic
- B. Category
- C. String builder
- D. Lookup

Q29. True or False: Disabling an OOTB integration pipeline means logs from that source will no longer be ingested.

- A. True
- B. False

Q30. An incoming log: `{"ts":"2026-05-03T14:22:01Z","level":"error","msg":"db down","@http.status_code":500}`. Without ANY custom pipeline, which is TRUE?

- A. `@http.status_code` is ignored because it starts with `@`.
- B. `level` is auto-promoted to `status:error`, `ts` is auto-promoted to `timestamp`, `msg` becomes the displayed message via the auto Message remapper (when configured).
- C. `@http.status_code` becomes a tag.
- D. The log is rejected as malformed JSON because of the `@` in a key.

Domain 3 — Quiz (30 hard questions) — Answer Key & Explanations

Q1. Answer: A

If the Status is correct, the pipeline IS matching and `@level` IS being extracted. The Date remapper consumes `@timestamp` — if the Grok pattern misses the date capture group, `@timestamp` is empty and the Date remapper does nothing (Datadog falls back to ingest time). Inspect the Grok parser's sample log preview to confirm `@timestamp` is captured.

Doc: docs.datadoghq.com/logs/log_configuration/processors/?tab=ui#date-remapper

Q2. Answer: D

The Grok syntax is `%{MATCHER:EXTRACT[:FILTER]}` — the leading `%` is required. Without `%`, `{integer:user_id}` is a literal pattern in the regex, not a matcher.

Doc: docs.datadoghq.com/logs/log_configuration/parsing/

Q3. Answer: B

Logs flow through EVERY matching pipeline in order — pipelines do not short-circuit. This is different from indexes, where logs go to the FIRST matching index only. Be cautious about declaring overlapping filters: each match adds processing cost.

Doc: docs.datadoghq.com/logs/log_configuration/pipelines/

Q4. Answer: B

`nullIf("...")` returns null instead of the matched value if the extracted text equals the argument. NGINX/Apache write `-`` for missing fields; this filter cleans those up so they aren't indexed as the literal string `-``.

Doc: docs.datadoghq.com/logs/log_configuration/parsing/?tab=filters#filter

Q5. Answer: C

There is no 'Regex Replace processor' in Datadog log pipelines. To mutate `message`, use a Message remapper or string-builder pattern. Grok, User-Agent, GeoIP, Arithmetic, URL parser, Category, Lookup, Reference table, Trace remapper, and the remappers (date/status/service/message/attribute) are the canonical processors.

Doc: docs.datadoghq.com/logs/log_configuration/processors/

Q6. Answer: B

The Category processor creates a new attribute whose value is determined by which range condition the input matches — ideal for latency bucketing. Arithmetic computes a new number; Lookup translates one value to another via a table; String builder concatenates.

Doc: docs.datadoghq.com/logs/log_configuration/processors/?tab=ui#category-processor

Q7. Answer: B

Standard attribute remappers map your fields into Datadog's standard `@http.*`, `@network.*`, `@error.*` namespaces — so OOTB dashboards, analytics, and APM-log correlation work. Other options require Arithmetic, GeoIP parser, or other processors.

Doc: docs.datadoghq.com/logs/log_configuration/attributes_naming_convention/

Q8. Answer: B

Integration pipelines are read-only and trigger on the `source` value (not `service`). Datadog ships hundreds — NGINX, Apache, Kafka, Postgres, etc. You can disable them or clone them and edit the clone, but you can't edit the originals.

Doc: docs.datadoghq.com/logs/log_configuration/pipelines/?tab=source

Q9. Answer: B

Narrow filters mean fewer logs are evaluated by each processor, lowering CPU at scale and making the pipeline list a clear map of what runs where. There's no 10-pipeline limit, and Datadog does not auto-optimize a single `` pipeline into something cheaper.

Doc: docs.datadoghq.com/logs/log_configuration/pipelines/

Q10. Answer: B

Datadog auto-promotes `level` AND `severity` (and a couple of other variants) from JSON to `status`. Common case values are recognized (`ERROR` → `error`). For non-standard naming you'd add a Status remapper.

Doc: docs.datadoghq.com/logs/log_configuration/attributes_naming_convention/

Q11. Answer: B

Support rules are reusable named patterns (like macros) referenced by match rules. For example, define `\_user\_id` % {integer:user_id} once and use `%{\_user\_id}` inside several match rules.

Doc: docs.datadoghq.com/logs/log_configuration/parsing/?tab=matchers#advanced-settings

Q12. Answer: B

Processors run sequentially in declaration order. Re-ordering is one of the most common fixes for parsing bugs (the date remapper must run after the grok that creates the timestamp field).

Doc: docs.datadoghq.com/logs/log_configuration/processors/

Q13. Answer: B

The Trace ID remapper exists exactly for custom log formats where the trace ID is in a non-standard field. It tells Datadog 'this attribute IS the trace ID' so the Trace side-panel can link logs to traces.

Doc: docs.datadoghq.com/logs/log_configuration/processors/?tab=ui#trace-id-remapper

Q14. Answer: B

`scale(N)` multiplies the matched number by N. `1500 × 0.001 = 1.5`. Common use: converting microseconds to milliseconds or bytes to KB during parsing.

Doc: docs.datadoghq.com/logs/log_configuration/parsing/?tab=filters

Q15. Answer: A

Lookup processors are inline (defined per processor, suitable for small static mappings). Reference Tables are uploaded CSVs (or pushed via API) that can be shared across many pipelines and re-versioned independently. For thousands of entries, use Reference Tables.

Doc: docs.datadoghq.com/integrations/guide/reference-tables/

Q16. Answer: B

Datadog supports a single level of nesting. The outer pipeline acts as a gate via its filter; the inner pipeline's processors only see logs that already matched the outer filter. Useful for grouping environment-specific or service-family transformations.

Doc: docs.datadoghq.com/logs/log_configuration/pipelines/

Q17. Answer: A

Datadog auto-detects `host` in JSON logs and uses it as the reserved attribute. This is essential for HTTP-intake logs where there is no Agent to inject the hostname (e.g., Lambda → Forwarder → Datadog).

Doc: docs.datadoghq.com/logs/log_configuration/attributes_naming_convention/#reserved-attributes

Q18. Answer: B

Measures must be numeric. The Arithmetic processor produces a numeric attribute that can be declared as a measure (with a unit) and used in sum/avg/percentile aggregations. Strings become facets, not measures.

Doc: docs.datadoghq.com/logs/explorer/facets/#units

Q19. Answer: B

The URL parser explodes `@http.url` into `url_details.host`, `url_details.path`, `url_details.queryString.*`, etc. Aggregating on `@http.url_details.path` (and grouping order IDs together via a downstream Grok or Category) gives clean route-level analytics without exploding cardinality.

Doc: docs.datadoghq.com/logs/log_configuration/processors/?tab=ui#url-parser

Q20. Answer: A

The Service remapper sets the reserved `service` attribute from an existing log attribute — useful when the Agent's `service:` wasn't set correctly or when the same Agent collects logs from multiple services emitting their own `service` field.

Doc: docs.datadoghq.com/logs/log_configuration/processors/?tab=ui#service-remapper

Q21. Answer: A

The Message remapper sets `message` (the headline shown in the Log Explorer list view) from a chosen attribute. Common use: a JSON log has `msg` or `error.message` that's more useful than the raw concatenation.

Doc: docs.datadoghq.com/logs/log_configuration/processors/?tab=ui#log-message-remapper

Q22. Answer: B

The NGINX integration pipeline parses the first IP token (`$remote_addr`) into `@network.client.ip`. The GeoIP parser then enriches that IP with country/city. The Agent's host is the server's hostname, not this field.

Doc: docs.datadoghq.com/integrations/nginx/

Q23. Answer: B

Parse first to create attributes (Grok), then promote them to reserved attributes (Date, Status), then enrich (GeoIP, User-Agent, Category). The Date/Status order is somewhat flexible but conventionally Date is first because timestamp is the most-relied-on reserved field downstream.

Doc: docs.datadoghq.com/logs/log_configuration/processors/

Q24. Answer: A

`:lowercase` is a transformation filter — it converts the captured value to lowercase before storing. This is useful to normalize `INFO` / `Info` / `info` into a single `info` value for consistent Status remapping.

Doc: docs.datadoghq.com/logs/log_configuration/parsing/?tab=filters

Q25. Answer: C

`log_id` is not a Datadog-recognized JSON key. The auto-promoted set covers `host`, `service`, `source`, `status`, `message`, `timestamp`, plus aliases (`level`/`severity` → `status`, `date` variants → `timestamp`).

Doc: docs.datadoghq.com/logs/log_configuration/attributes_naming_convention/

Q26. Answer: B

Aggregations like avg/sum/percentile require the attribute to be declared a Measure, not just a Facet. A measure carries a unit and is the only attribute type usable for numeric compute in the Analytics tab.

Doc: docs.datadoghq.com/logs/explorer/facets/#units

Q27. Answer: A

The filter requires ``service`` to be billing OR orders AND ``env`` to be prod. Only the first option meets both conditions. The implicit AND between space-separated terms binds tightly.

Doc: docs.datadoghq.com/logs/explorer/search_syntax/

Q28. Answer: C

The String builder processor concatenates attributes (with literal text) into a new string attribute. Arithmetic is numeric only; Category creates discrete buckets; Lookup translates values.

Doc: docs.datadoghq.com/logs/log_configuration/processors/?tab=ui#string-builder-processor

Q29. Answer: B

FALSE. Pipelines run AFTER ingestion. Disabling a pipeline just skips its processing — logs still flow through ingestion, archives, metrics, indexes. They will simply not get OOTB parsing applied.

Doc: docs.datadoghq.com/logs/log_configuration/pipelines/

Q30. Answer: B

Auto-JSON parsing promotes ``level`` → ``status``, recognized timestamp fields → ``timestamp``, and you'd ordinarily want a Message remapper for ``msg`` → ``message`` (a tiny custom pipeline). ``@http.status_code`` is stored as-is (the leading ``@`` is convention in JSON keys passed through), and JSON keys with ``@`` are legal.

Doc: docs.datadoghq.com/logs/log_configuration/attributes_naming_convention/

Domain 4 — Log Searching & Filtering

Live vs. Explorer · Search Syntax · Explorer Functionality

Domain 4 is highly practical: it asks whether you can write a correct query for a real-world scenario. Expect questions about the exact difference between Live Tail and the Log Explorer (15-min window vs index retention; all ingested vs indexed only), the search syntax (including range queries with `[ ]` vs `{ }`, wildcards, escaping), facets vs measures, Patterns, Transactions, and Saved Views.

4.1 Live Tail vs Log Explorer

	Live Tail	Log Explorer
Time range	Last ~15 min, streaming	Index retention (3-90 days)
Scope	ALL ingested logs (even excluded ones)	Indexed logs only
Search	Filter on the live stream	Full faceted search
Aggregations	No (list view only)	Yes (timeseries, top list, table, tree map, pie, geomap)
Saved views	No	Yes
Cost	No search cost	Search cost via index retention

Exam trap. A question asking how to verify a BRAND-NEW service is sending logs almost always wants `Live Tail` — because the logs may not be in any index yet (and might be excluded from indexing). The Explorer would show nothing even though logs ARE arriving.

4.2 Search syntax cheat sheet

- Tags and reserved attributes (host, service, source, status, message, timestamp) DO NOT need `@`.
- Custom and standard attributes (@http.*, @network.*, @your.attr) REQUIRE `@`.
- Boolean operators (AND, OR, NOT) MUST be uppercase. Lowercase `and` is parsed as a free-text token.
- Special characters in values (`+ - = & | > < ! ( ) { } [ ] ^ " ~ \* ? : \ /`) need `\"` escaping or quoting.

```
# Tags / reserved attributes (no `@`)  
service:checkout env:prod host:web-01 status:error source:nginx
```

```
# Attributes (require `@`)  
@http.status_code:500  
@http.url:/api/v1/orders  
@duration:>1000000000
```

```
# Ranges: [a TO b] inclusive, {a TO b} exclusive  
@http.status_code:[500 TO 599]  
@duration:{1000 TO *}
```

```
# Boolean operators — UPPERCASE only  
service:checkout AND status:error  
status:(error OR warn)  
service:web NOT @http.status_code:200  
-status:debug          # ` ` is shorthand for NOT
```

```
# Wildcards
```

```

service:payment-*      # trailing — fine
host:web-?             # single-char
*error                 # leading — slow, avoid

# Existence / nullity
@usr.id:*              # attribute exists
-@usr.id:*              # attribute MISSING
@usr.id:""             # empty string (≠ missing)

# Free text matches the `message` attribute by default
"connection refused"

```

4.3 Anatomy of the Explorer

- Time range selector (top right) — relative or absolute, with Live Tail option.
- Search bar — free text + facet syntax.
- Facets sidebar (left) — clickable facets and measures; histograms for numerics, top-N for strings.
- Visualization tabs — List, Timeseries, Top List, Table, Tree Map, Pie Chart, Geomap, Patterns, Transactions.
- Right side panel — opens on log click; shows full attributes, related traces/metrics, `View in context`.

4.4 Saved Views

A Saved View stores: the query, the time range (relative preserved as relative, absolute as fixed), the chosen visualization, and the selected facets. The URL takes the form `?saved\_view=<id>` and can be combined with `&query=...` to layer an ad-hoc filter on top.

Use for team conventions. Pin frequently-used views (per-service error feeds, per-region dashboards) to a team folder. Reference them in dashboard widgets and monitor messages so links don't go stale.

4.5 Facets vs Measures

Reserved attributes and standard attributes are auto-faceted. CUSTOM JSON keys are NOT — you must explicitly create the facet.

Type	Use case	Example
Tag facet	Filter by tag values in the sidebar	env, service, team
String facet	Filter by attribute string values	@usr.id, @http.url_details.path
Numeric measure	Aggregate (sum/avg/percentile) — needs a unit	@duration (nanoseconds), @http.response_size (bytes)

Subtle point. Free-text search by `@attr:value` works on ANY attribute, even unfaceted ones. Faceting is required only to use the attribute in the sidebar, as a group-by, or in ranges/computes. Numeric aggregations (avg, sum, p95) require declaring the attribute as a Measure, not just a facet.

4.6 Patterns

The Patterns view auto-clusters logs that share a similar text structure (hiding variable parts like IDs and timestamps). Use it to spot dominant log shapes you can drop, surface rare new error patterns, and build exclusion filters with one click. Patterns work on the `message` attribute by default.

4.7 Transactions

Transactions group related logs by a shared attribute (e.g., `@request\_id`, `@session\_id`) into a single timeline. You set a grouping attribute and a maximum duration window — logs further apart get separated into different transactions. Optionally add a start condition and an end condition. Useful when you don't have full APM tracing but a request_id flows through your logs.

4.8 Visualization group-by limits

Visualization	Max group-by dimensions
Top List	1
Timeseries	4
Table	4
Tree Map	4
Pie Chart	4
Geomap	4 (requires GeoIP-populated attribute)

4.9 Exam takeaways

- Live Tail = last 15 min, ALL ingested logs. Explorer = indexed only.
- Tags / reserved attributes are queried WITHOUT `@`; custom attributes need `@`.
- Range syntax: `[a TO b]` inclusive, `{a TO b}` exclusive; `` is the open bound.
- Trailing/mid-token wildcards are OK; leading wildcards (`\*error`) are slow.
- Boolean operators must be UPPERCASE.
- Patterns cluster similar logs; Transactions group by a shared correlation attribute.

Domain 4 — Quiz (28 hard questions)

Search-syntax precision matters. Internalize when to use @ and the range/wildcard rules.

Q1. A user reports: 'I added a new attribute `@tenant.id` and I can search `@tenant.id:abc-123` to find logs, but I can't group by `tenant.id` in the Analytics tab.' The cause is:

- A. The attribute needs to be added as a FACET; only faceted attributes can be used in the sidebar and as group-by.
- B. The log volume is too low.
- C. Only standard attributes can be grouped.
- D. Group-by only works on tags, not attributes.

Q2. What is the EXCLUSIVE-range syntax in Datadog log search?

- A. `@x:[a TO b]` is exclusive on both ends.
- B. `@x:{a TO b}` is exclusive on both ends; brackets are inclusive.
- C. `@x:(a TO b)` is exclusive on both ends.
- D. There is no exclusive range; use boolean operators instead.

Q3. Which query finds logs from any host whose name starts with `web-` AND has `@http.status_code` of 500–599?

- A. `host:web-* @http.status_code:[500 TO 599]`
- B. `host:"web-*" @http.status_code:500..599`
- C. `@host:web-* @http.status_code:>=500`
- D. `host:web AND @http.status_code:5xx`

Q4. A team wants to verify that a brand-new shipping service is sending logs to Datadog. They've just deployed the Agent. Where do they look FIRST?

- A. Open the Log Explorer and search `service:shipping`.
- B. Open Live Tail and filter `service:shipping`.
- C. Check the Indexes page to see if the service appears.
- D. Wait an hour, then check the Log Explorer.

Q5. Which is TRUE about leading wildcards in queries?

- A. `**error*` is faster than `error*`.
- B. `**error*` is slow and discouraged; `error*` is fine.
- C. Leading wildcards are not supported at all.
- D. Wildcards are only allowed inside attribute values, not tags.

Q6. Which Saved View component is NOT preserved when you share its URL?

- A. The query string.
- B. The selected visualization (List, Timeseries, Patterns).
- C. Per-user UI preferences like log line density.
- D. The time range (relative or absolute).

Q7. True or False: Free-text search (e.g., `"connection refused"`) matches against the ``message`` field by default.

- A. True
- B. False

Q8. Which query is logically INCORRECT?

- A. ``service:checkout AND status:error``
- B. ``service:(checkout OR billing) NOT status:info``
- C. ``service:checkout and status:error``
- D. ``-status:debug``

Q9. Which is FALSE about Patterns?

- A. Patterns cluster logs that share a similar text structure.
- B. Patterns operate on the ``message`` attribute by default.
- C. Patterns can be one-clicked into an exclusion filter.
- D. Patterns can be configured to run only on logs in archives.

Q10. A ``Transactions`` visualization groups logs by:

- A. Service automatically.
- B. A correlation attribute you pick (e.g., ``@request_id``) plus a time window.
- C. Anomaly score (Watchdog-derived).
- D. Pipeline-of-origin.

Q11. What's the difference between ``source`` and ``service`` in a log search?

- A. They are synonyms — Datadog stores both for backward compatibility.
- B. ``source`` is the technology emitting the log (e.g., `nginx`, `java`, `python`); ``service`` is the logical service name (e.g., `checkout-api`).
- C. ``source`` is set by pipelines; ``service`` is set by the Agent.
- D. ``source`` is for tags; ``service`` is for attributes.

Q12. A query ``@usr.id:*`` returns 1.2M logs. What does this query mean?

- A. Logs where the user ID equals literal ``*``.
- B. Logs where the ``@usr.id`` attribute exists (any value).
- C. Logs from all users (no filtering).
- D. Logs where the value is a wildcard string.

Q13. True or False: Searching ``@http.url:"/v1/users"`` returns logs where the URL CONTAINS that substring.

- A. True
- B. False

Q14. Which is the BEST single-line Explorer query to find ALL errors AND warnings from checkout in prod, EXCLUDING healthcheck noise?

- A. ``service:checkout env:prod status:(error OR warning) -@http.url:/health``
- B. ``service:checkout AND env=prod AND status:(error,warning) AND NOT @http.url:/health``
- C. ``service:checkout & env:prod & status:[error TO warning]``
- D. ``(service:checkout env:prod status:error) OR (service:checkout env:prod status:warning) NOT health``

Q15. Which is TRUE about cardinality limits on facets?

- A. Datadog enforces a hard limit of 10k distinct values per facet.
- B. Very high-cardinality attributes (raw user IDs, full URLs with query strings) make poor facets — they slow the sidebar and offer little exploration value. Datadog recommends storing them as attributes but not faceting.
- C. Facets are automatically pruned to top-1000 values.
- D. Facets have no cardinality concerns.

Q16. A SOC analyst wants to set up a saved view scoped to ``host:web-* status:error`` for ongoing monitoring. Where does the URL parameter ``&saved_view=`` come from?

- A. It's auto-generated when you save the view; sharing the URL allows others to load it.
- B. Generated only after pinning to the team folder.
- C. It is the saved view's display name URL-encoded.
- D. Generated only for paid plans.

Q17. What does the Geomap visualization require to render a country-level heatmap?

- A. Free-text search of ``country:*``
- B. A group-by on ``@network.client.geoip.country.iso_code`` (or city/region), populated by a GeoIP parser.
- C. Manual upload of a GeoJSON file.
- D. A Reference Table mapping IPs to countries.

Q18. Which is TRUE about ``View In Context``?

- A. It opens a stream of all logs from the same ``host`` and ``service`` around the selected log's timestamp.
- B. It re-runs the pipeline for that log.
- C. It links the log to its associated APM trace.
- D. It opens the source code of the application emitting the log.

Q19. Top List visualization supports up to how many group-by dimensions?

- A. 1
- B. 2
- C. 4
- D. 10

Q20. Which is FALSE about facet creation?

- A. Reserved attributes are auto-faceted.
- B. Standard attributes are auto-faceted.
- C. Custom JSON keys become facets automatically.
- D. You can group facets under a custom 'Group' name in the sidebar.

Q21. A team wants to alert on logs that appear in Live Tail but are NEVER indexed. Which monitor type works?

- A. A standard log monitor on the `main` index.
- B. An Anomaly log monitor.
- C. A Live (Live Tail) monitor — watches the live stream regardless of indexing.
- D. A Watchdog detection rule.

Q22. Which character requires escaping in a search value (using `\\`) but does NOT require quoting?

- A. space
- B. colon `:`
- C. comma `,`
- D. letter

Q23. Which is the best query to find logs from any of `service:billing`, `service:orders`, or `service:checkout` with status either `error` or `warning`?

- A. `service:(billing OR orders OR checkout) status:(error OR warning)`
- B. `service:billing|orders|checkout status:error|warning`
- C. `service:billing,orders,checkout status:error,warning`
- D. `service:[billing TO checkout] status:[error TO warning]`

Q24. Live Tail's ~15-minute window is best described as:

- A. A rolling buffer of recently ingested logs; it does NOT serve as historical search.
- B. A configurable retention window up to 24h.
- C. The shortest of the indexes' retention values.
- D. Equivalent to the smallest Saved View time range.

Q25. Which visualization is INAPPROPRIATE for showing the share of total errors per service?

- A. Pie Chart
- B. Tree Map
- C. Geomap
- D. Top List

Q26. True or False: A query in the Log Explorer can include up to 4 group-by dimensions in a Table visualization.

- A. True
- B. False

Q27. Which is the BEST way to share a custom log search with a teammate without forcing them to recreate it?

- A. Screenshot the query and message it.
- B. Save it as a Saved View and share the URL (with `?saved_view=<id>`).
- C. Copy the query string and paste it into a Slack DM.
- D. Export the entire search result set to CSV.

Q28. Which two query elements work WITHOUT the `@`` prefix?

- A. tags (e.g., `env:prod`) and reserved attributes (e.g., `status:error`)
- B. tags and standard attributes
- C. reserved attributes and measures
- D. facets and measures

Domain 4 — Quiz (28 hard questions) — Answer Key & Explanations

Q1. Answer: A

Free-text `@key:value` search works on any attribute. But sidebar listing, range filtering, and group-by all require the attribute to be a Facet (or a Measure for numeric aggregations). Add it via right-click → 'Create facet'.

Doc: docs.datadoghq.com/logs/explorer/facets/

Q2. Answer: B

Datadog (like Lucene-family syntax) uses `[...]` for inclusive and `{...}` for exclusive bounds. They can be mixed: `[a TO b]` is inclusive on `a` and exclusive on `b`. `*` is the open bound.

Doc: docs.datadoghq.com/logs/explorer/search_syntax/

Q3. Answer: A

`host:web-*` is a wildcard on a tag (no `@`); the range query uses `[500 TO 599]`. The other syntaxes are incorrect for Datadog: there's no `..` operator, `host` is a tag not an attribute, and `5xx` isn't valid syntax.

Doc: docs.datadoghq.com/logs/explorer/search_syntax/

Q4. Answer: B

Live Tail shows ALL ingested logs in the last ~15 min, even logs that aren't indexed yet (and even those excluded from indexing). It's the fastest way to confirm 'logs are arriving from this service'. Explorer only shows indexed logs.

Doc: docs.datadoghq.com/logs/explorer/live_tail/

Q5. Answer: B

Leading wildcards (`*error`) force scanning every value — they're slow and Datadog discourages them. Trailing or mid-token wildcards (`error*`, `we?-01`) leverage indexed prefixes and are fine.

Doc: docs.datadoghq.com/logs/explorer/search_syntax/?tab=unifiedservicetagging#wildcards

Q6. Answer: C

Saved Views preserve query, time range (relative is preserved as relative), visualization, and selected facets. Per-user UI prefs (line density, column widths) live in the user's local profile, not in the view.

Doc: docs.datadoghq.com/logs/explorer/saved_views/

Q7. Answer: A

TRUE. Quoted free-text matches `message` by default. To search other attributes, scope with `@attr:"..."`. This is why the `message` field is so important — auto-JSON keeps the original message text in there for free-text discoverability.

Doc: docs.datadoghq.com/logs/explorer/search_syntax/

Q8. Answer: C

Boolean operators MUST be uppercase: `AND`, `OR`, `NOT`. Lowercase `and` is parsed as a free-text token, so `service:checkout and status:error` actually means `service:checkout` AND the word `and` AND `status:error` — a very different query. `-` as a NOT shortcut is allowed.

Doc: docs.datadoghq.com/logs/explorer/search_syntax/

Q9. Answer: D

Patterns run against the indexed Log Explorer (live, on-demand). They don't run against archives — to analyze archived logs you'd need to rehydrate first. The other three are accurate.

Doc: docs.datadoghq.com/logs/explorer/patterns/

Q10. Answer: B

Transactions group logs by a shared correlation attribute over a configurable max duration. Useful when full APM tracing isn't available but a `request_id` is present across services.

Doc: docs.datadoghq.com/logs/explorer/transactions/

Q11. Answer: B

`source` and `service` are distinct reserved attributes. `source` triggers integration pipelines and identifies the technology; `service` ties the log to APM/RUM via Unified Service Tagging. Both can be queried without `@`.

Doc: docs.datadoghq.com/getting_started/tagging/unified_service_tagging/

Q12. Answer: B

`@attr:\*` is the existence operator — match any log where the attribute is present. To search the opposite (attribute missing), use `~@usr.id:\*` or `NOT @usr.id:\*`. The literal `\*` value would need quoting.

Doc: docs.datadoghq.com/logs/explorer/search_syntax/

Q13. Answer: B

FALSE. Quoted values are matched as exact strings (phrase). To do contains-match, use wildcards:

`@http.url:\*\\v1\\users\*` (mind the slashes escaping). For sub-token searches you'd typically faceting and use facet-based filtering, or parse the URL into path components.

Doc: docs.datadoghq.com/logs/explorer/search_syntax/

Q14. Answer: A

Implicit AND between space-separated terms, parenthesized OR for status, `~` for NOT. Option B uses `=` (invalid; use `:`), `;` (invalid), and capital `AND` (valid but the syntax has other errors). C uses `&` (invalid). D is functionally OK but inefficient and verbose; `health` alone matches the word, not `@http.url`.

Doc: docs.datadoghq.com/logs/explorer/search_syntax/

Q15. Answer: B

There's no hard 10k limit, but high-cardinality facets degrade UI responsiveness and explore poorly (a histogram of millions of unique values is meaningless). Keep these as attributes (free-text searchable) but skip faceting.

Doc: docs.datadoghq.com/logs/explorer/facets/

Q16. Answer: A

When you save a view, Datadog assigns it a numeric ID and the URL includes `?saved\_view=<id>`. Sharing this URL loads the same query/visualization/time-range/facets for any user with access.

Doc: docs.datadoghq.com/logs/explorer/saved_views/

Q17. Answer: B

Geomap reads ISO country codes from the standard `@network.client.geoip.\*` attributes, populated by Datadog's GeoIP parser. You'd add a GeoIP parser in your pipeline that converts `@network.client.ip` to country/city codes, then group by `iso\_code`.

Doc: docs.datadoghq.com/logs/log_configuration/processors/?tab=ui#geoip-parser

Q18. Answer: A

View In Context is a powerful pivot: from a single log, see ALL logs from the same `host` + `service` ± a small time window. Useful for tracing stack-trace causes without APM. The APM trace link is the separate 'View Trace' button (and depends on `dd.trace\_id`).

Doc: docs.datadoghq.com/logs/explorer/

Q19. Answer: A

Top List shows top-N by a single dimension. For multi-dimensional cross-tabs, use Table or Timeseries (up to 4 dimensions).

Doc: docs.datadoghq.com/logs/explorer/analytics/

Q20. Answer: C

Custom JSON keys are NOT auto-faceted — you must explicitly create the facet via the UI or API. Reserved and standard attributes ARE auto-faceted. You can also organize facets into groups for sidebar tidiness.

Doc: docs.datadoghq.com/logs/explorer/facets/

Q21. Answer: C

Standard log monitors query indexed logs. The Live (Live Tail) monitor is specialized for the live stream — it triggers when a query matches any log being ingested, independent of indexing. Less commonly tested but worth knowing.

Doc: docs.datadoghq.com/monitors/types/log/

Q22. Answer: B

Colons are field separators in search syntax (`key:value`), so a literal `:` inside a value needs `\:` or quoting. Spaces always require quoting; commas have no special meaning; letters are fine.

Doc: docs.datadoghq.com/logs/explorer/search_syntax/

Q23. Answer: A

Parenthesized OR is the correct multi-value syntax. `|`, `,`, and ranges on strings are not valid. Datadog's syntax matches Lucene-style boolean grouping.

Doc: docs.datadoghq.com/logs/explorer/search_syntax/

Q24. Answer: A

Live Tail is a streaming view, not a backing store. The ~15-minute window exists because logs flow through it in near-real-time. It's not configurable in length and isn't related to index retention.

Doc: docs.datadoghq.com/logs/explorer/live_tail/

Q25. Answer: C

Geomap requires geographic data (country/city/ISO codes). For proportional share by service, Pie, Tree Map, and Top List are all sensible (each makes different trade-offs in visual clarity).

Doc: docs.datadoghq.com/logs/explorer/analytics/

Q26. Answer: A

TRUE. Table, Timeseries, Tree Map, Pie Chart, and Geomap all support up to 4 group-by dimensions. Top List supports only 1.

Doc: docs.datadoghq.com/logs/explorer/analytics/

Q27. Answer: B

Saved Views preserve the full search context (query, time range, viz, facets) and can be pinned to a team folder. The URL with `?saved\_view=<id>` loads the same view for any authorized user.

Doc: docs.datadoghq.com/logs/explorer/saved_views/

Q28. Answer: A

Tags (`env:`, `team:`, etc.) and reserved attributes (`host`, `service`, `source`, `status`, `message`, `timestamp`) are queried WITHOUT `@`. Everything else (standard `@http.\*` AND custom `@\*` attributes/measures) requires `@`.

Doc: docs.datadoghq.com/logs/explorer/search_syntax/

Domain 5 — Log Analysis

Filtering and Excluding · Aggregations · Visualizing

Domain 5 distinguishes WHERE filtering happens (Agent edge, pipelines, index inclusion, index exclusion, search-time) and the impact each has on volume, cost, and accessibility. Aggregations (compute + group-by) and the visualization picker round out the section.

5.1 Filtering vs Excluding — where each lives

Concept	Where	Effect
Search filter	Explorer query bar	Narrows what YOU see, doesn't change storage
Saved-view filter	Inside a Saved View	Pre-configured query that opens with the view
Index inclusion filter	Logs → Configuration → Indexes	Routes matching logs INTO the index (first match wins)
Index exclusion filter	Per index	Drops a PERCENTAGE after inclusion; not stored in index
Restriction query (RBAC)	Per index	Limits which roles can see which logs
Agent edge filter	log_processing_rules → exclude_at_match	Drops on the HOST before sending — saves ingestion

5.2 Indexes — inclusion, exclusion, retention

Three levers per index:

- Inclusion filter: query that defines what enters the index. Indexes evaluate in ORDER; a log goes into the FIRST matching index.
- Exclusion filters: multiple per index, each with a 0-100% drop rate. Excluded logs are still INGESTED, PARSED, ARCHIVED, and count toward metric generation — they just don't go into the searchable index.
- Retention: how long indexed logs are searchable. Buckets: 3, 7, 15, 30, 60, 90 days (plan-dependent).

Repeat after me. Exclusion filters drop from the INDEX, not from ingestion. To save ingestion cost, drop at the Agent edge.

Daily quota per index: a hard ceiling on indexed-event count per day. Logs above the quota continue to be ingested (and archived, metrics-generated) but are not indexed for the rest of the day.

5.3 Aggregations — turning logs into numbers

The Analytics tab and all non-list visualizations run aggregations. Pick up to 4 group-by dimensions (1 for Top List) and a compute.

Compute	Source	Description
count	any matching log	Number of logs
cardinality(<facet>)	any facet	Distinct values
sum / avg / min / max	measure	Standard math
median (p50)	measure	50th percentile
pcXX (p75, p95, p99)	measure	Configurable percentile

Measures vs facets. Numeric aggregations beyond count REQUIRE the attribute to be declared a MEASURE (with a unit). Faceting alone enables sidebar filtering but not aggregation.

5.4 Visualization picker

Visualization	Best for	Limits
List	Reading individual log lines	No aggregation
Timeseries	Trends over time	Up to 4 group-by
Top List	Top N by metric	1 group-by only
Table	Multi-column comparison	Up to 4 dimensions, multiple computes
Tree Map	Proportional contribution	1-4 dimensions
Pie Chart	Share of total	1-4 dimensions; clutters above ~6 slices
Geomap	Geographic distribution	Group by @network.client.geoip.country.iso_code
Patterns	Auto-cluster log shapes	No group-by
Transactions	Group by correlation attr over time window	1 grouping attr

5.5 Drilldown patterns

- Metric monitor → Related Logs (preloads service tag + time window).
- APM trace → Logs tab (matches by `dd.trace\_id`).
- Error log → View Trace button (requires `dd.trace\_id` on the log).
- View In Context — stream of all logs from the same host AND service around the chosen log's timestamp.
- Watchdog Insights — auto-detected unusual patterns; no setup required.

5.6 The filter-early principle

Aggregations run against INDEXED logs only. If a log isn't in any index (excluded, archived only), it won't appear in Explorer aggregations. To analyze excluded logs, rehydrate the relevant archive into a temporary index. And always pre-filter your query tightly before grouping — querying `\*` then grouping by `@usr.id` is much slower than `service:checkout status:error` then grouping.

5.7 Exam takeaways

- Exclusion filter ≠ ingestion drop. Excluded logs still pay ingestion, still go to archives, still drive metrics.
- An index has ONE inclusion filter, MULTIPLE exclusion filters (each 0-100%), and a retention setting.
- Indexes match the FIRST inclusion filter that hits; pipelines match EVERY filter that hits.
- Top List supports 1 group-by; everything else supports up to 4.
- Percentiles (p95, p99) require declaring the attribute as a MEASURE with a unit.
- To see excluded logs, use Live Tail (within 15 min) or rehydrate from archive.

Domain 5 — Quiz (22 hard questions)

Know where every kind of filtering lives — Agent edge vs pipeline vs index inclusion vs index exclusion vs search-time.

- Q1.** Your team excludes healthcheck logs from the ``main`` index at 100%. Two weeks later, you need to know how many healthchecks failed yesterday. Where do you find them?
- A. They're gone — excluded logs are dropped entirely.
 - B. Live Tail (if you're still within the 15-min window).
 - C. Archives — rehydrate to a temporary index for the failed window, then search.
 - D. The pipeline diagnostics screen.
- Q2.** A team configures 3 indexes in this order: ``prod-errors`` (filter ``status:error env:prod``), ``prod-all`` (``env:prod``), ``default`` (``*``). A log with ``service:foo env:prod status:error`` lands in which index?
- A. ``prod-errors`` — the first matching index wins.
 - B. All three — logs go to every matching index.
 - C. ``default`` — the broader catch-all takes precedence.
 - D. None — ``status:error`` conflicts with ``env:prod``.
- Q3.** An index has an exclusion filter: ``source:nginx @http.url:/health`` with 80% sample. What happens?
- A. All matching logs are dropped from the index.
 - B. 80% of matching logs are dropped from the index; 20% are still indexed (and the rest of the flow is unaffected).
 - C. 80% of matching logs are dropped from ingestion entirely.
 - D. Matching logs are also excluded from archives.
- Q4.** Which compute aggregation requires the source attribute to be declared as a MEASURE (not just a facet)?
- A. ``count``
 - B. ``cardinality(@usr.id)``
 - C. ``p95(@duration)``
 - D. All of the above
- Q5.** What is the difference between an index INCLUSION filter and a search filter (in the Explorer query bar)?
- A. None — they use the same query and apply identically.
 - B. Inclusion is permanent and routes logs into the index; search filter only narrows what YOU see for this query.
 - C. Inclusion runs after exclusion filters; search filter runs before.
 - D. Search filter can be regex; inclusion can only be exact-match.
- Q6.** A team needs to alert on the count of unique users hit by errors per service per minute. The correct compute is:
- A. ``sum(@usr.id)``
 - B. ``cardinality(@usr.id)``
 - C. ``count`` group-by `@usr.id` and service

D. ``distinct(@usr.id)``

Q7. Which retention values are configurable on an index (standard plan)?

- A. Any integer from 1–365 days.
- B. 3, 7, 15, 30, 60, 90 days (subject to plan).
- C. Only 30 or 90 days.
- D. Retention is set by Datadog and cannot be customized.

Q8. Which view supports analyzing logs older than the index retention?

- A. Live Tail (in the historical mode).
- B. Patterns (auto-clusters across all time).
- C. Rehydrate from Archive into a temporary index.
- D. Notebooks (which read from archives by default).

Q9. True or False: A daily index quota and an exclusion filter are mutually exclusive — you can use only one per index.

- A. True
- B. False

Q10. A team wants to drop ALL ``status:debug`` logs from production to save on ingestion. The BEST place is:

- A. Index exclusion filter at 100% on ``status:debug env:prod``.
- B. Agent ``log_processing_rules`` ``exclude_at_match`` on debug-level patterns.
- C. Pipeline processor that strips debug logs.
- D. Archive lifecycle rule deleting after 1 day.

Q11. Which visualization aligns with the question 'what proportion of total error volume is each service contributing?'

- A. Timeseries
- B. Pie Chart or Tree Map
- C. Top List
- D. Geomap

Q12. Group-by + Compute can also work on:

- A. Tags (e.g., ``service``, ``env``)
- B. Reserved attributes (e.g., ``status``)
- C. Standard and custom facets
- D. All of the above

Q13. A monitor query computes ``cardinality(@usr.id)`` over 5 minutes for ``service:checkout``. Logs are excluded from ``main`` index for ``env:dev``. Will the monitor fire for dev errors?

- A. Yes — monitors run on all ingested logs.

- B. No — monitors run on indexed logs; excluded logs are invisible.
- C. Only if anomaly detection is enabled.
- D. Only after rehydration.

Q14. Sampling at 20% in an exclusion filter means:

- A. 20% of matching logs ARE indexed; 80% are excluded.
- B. 20% of matching logs are EXCLUDED; 80% are indexed.
- C. 20% of all logs are sampled regardless of match.
- D. 20% of matching logs are sent to archives.

Q15. True or False: A monitor with `Multi Alert` enabled on `service` will create one underlying monitor for each unique service value, sharing the same query.

- A. True
- B. False

Q16. Which is FALSE about Watchdog Insights for Logs?

- A. It surfaces unusual log patterns automatically.
- B. It requires the user to enable specific detection rules.
- C. It's available from the Logs landing page.
- D. It runs continuously without configuration.

Q17. Which is the closest equivalent of 'how does this hour's error rate compare to the same hour last week?'

- A. Anomaly monitor
- B. Change alert monitor
- C. Threshold monitor
- D. Composite monitor

Q18. An analyst sees `count\_per\_minute` for a `status:error` query gradually rising over 7 days. The most powerful tool to confirm whether the rise is statistically anomalous:

- A. Threshold monitor with the current count.
- B. Anomaly monitor with seasonal awareness.
- C. Manual visual inspection on Timeseries.
- D. Cardinality monitor on `@usr.id`.

Q19. Which is TRUE about logs-to-metrics?

- A. Only counts can be generated, never distributions of numeric attributes.
- B. Up to 10 tag dimensions per metric.
- C. Only logs in indexes can drive metric generation.
- D. Generated metrics are billed as ingested logs.

Q20. An analyst opens an error log and uses `View In Context`. What set of logs does Datadog show?

- A. All logs sharing the same `dd.trace\_id`.
- B. All logs from the same `host` and `service`, in a small time window around the chosen log.
- C. All errors across the org.
- D. Only logs from the same process ID.

Q21. Which is FALSE about exporting an Analytics result?

- A. It can be exported to a Dashboard widget.
- B. It can be exported to a Notebook.
- C. It is automatically saved as a Saved View.
- D. It can be sent to a Monitor (for some compute types).

Q22. Which is the MOST cost-efficient pattern for high-volume access logs that need only graphs (no individual-log search)?

- A. Index 100% of access logs with a 30-day retention.
- B. Drop access logs at the Agent edge.
- C. Exclude access logs from indexes at 100%, generate `requests\_per\_minute` and other metrics from them, archive for forensics.
- D. Forward access logs to S3 only — bypass Datadog.

Domain 5 — Quiz (22 hard questions) — Answer Key & Explanations

Q1. Answer: C

Excluded logs still get ingested, parsed, and archived. To search them post-hoc, rehydrate the relevant archive into a temporary index, then query as usual. Live Tail is only valid in the current ~15-min window.

Doc: docs.datadoghq.com/logs/log_configuration/rehydrating/

Q2. Answer: A

Indexes are evaluated in ORDER, and the FIRST matching inclusion filter receives the log. This is different from pipelines (which run for every match). Order indexes from most-specific to most-general.

Doc: docs.datadoghq.com/logs/log_configuration/indexes/

Q3. Answer: B

Exclusion percentage is a sampling rate. 80% means roughly 4 in 5 matching logs are not indexed; 1 in 5 IS indexed (useful for keeping a representative sample). Archives, metrics, and live tail are unaffected — exclusion is purely an index-level decision.

Doc: docs.datadoghq.com/logs/log_configuration/indexes/#exclusion-filters

Q4. Answer: C

`count` works on any matching log. `cardinality` works on any facet (string or numeric). Percentile (`p95`), avg, sum, min, max all require a numeric measure with a unit. Declare `@duration` as a Measure with unit nanoseconds first.

Doc: docs.datadoghq.com/logs/explorer/analytics/

Q5. Answer: B

Inclusion filter is the index's routing rule — it permanently decides which logs enter the searchable store. The Explorer query is a transient lens over already-indexed logs.

Doc: docs.datadoghq.com/logs/log_configuration/indexes/

Q6. Answer: B

Cardinality counts distinct values of a facet. `sum` requires numeric; `count` group-by user would explode rows; `distinct` is not a Datadog aggregator (use `cardinality`).

Doc: docs.datadoghq.com/logs/explorer/analytics/

Q7. Answer: B

Indexes have a small set of retention buckets: 3, 7, 15, 30, 60, 90 days (longer values are available on certain plans). Mix-and-match retention by separating logs into different indexes (e.g., 7-day debug, 90-day security).

Doc: docs.datadoghq.com/logs/log_configuration/indexes/#configure-indexes

Q8. Answer: C

Rehydration re-ingests archive content into a temporary index for the chosen window. Live Tail is current-only; Patterns runs against indexed data; Notebooks query indexes/metrics, not archives directly.

Doc: docs.datadoghq.com/logs/log_configuration/rehydrating/

Q9. Answer: B

FALSE. An index can have both: exclusion filters reduce volume by selective sampling, and the daily quota acts as a hard ceiling. Use them together for predictable cost (quota) + targeted noise reduction (filters).

Doc: docs.datadoghq.com/logs/log_configuration/indexes/

Q10. Answer: B

Exclusion filters reduce INDEX cost but you still pay for INGESTION. Dropping at the Agent edge saves both. Use this strategy especially for high-volume debug logs in prod that nobody investigates.

Doc: docs.datadoghq.com/agent/logs/advanced_log_collection/

Q11. Answer: B

Proportion-of-total is a part-of-whole comparison — Pie Chart or Tree Map. Timeseries shows trends; Top List shows rank; Geomap shows geography. Tree Map scales better when there are many services (Pie clutters above ~6 slices).

Doc: docs.datadoghq.com/logs/explorer/analytics/

Q12. Answer: D

Any indexed dimension that's either a tag, reserved attribute, or facet can be a group-by dimension. Numeric aggregations beyond count require a Measure.

Doc: docs.datadoghq.com/logs/explorer/analytics/

Q13. Answer: B

Standard log monitors run on indexed logs. If dev logs are excluded from the index they target, they won't drive the monitor. To alert on excluded logs, either index them, generate a custom metric, or use a Live monitor.

Doc: docs.datadoghq.com/monitors/types/log/

Q14. Answer: B

The percentage in an exclusion filter is the DROP rate. 20% exclusion means 20% are excluded from the index and 80% are retained. This is the opposite of what some intuit.

Doc: docs.datadoghq.com/logs/log_configuration/indexes/#exclusion-filters

Q15. Answer: A

TRUE. Multi Alert evaluates the query per group (per unique tag value) and alerts independently. This is how you say 'alert me per service' with a single monitor definition.

Doc: docs.datadoghq.com/monitors/configuration/?tab=thresholdalert#alert-grouping

Q16. Answer: B

Watchdog Insights for Logs runs WITHOUT explicit configuration. It auto-detects spikes, new error patterns, and latency anomalies and surfaces them in the Watchdog tab. Detection rules belong to Cloud SIEM, not Watchdog.

Doc: docs.datadoghq.com/watchdog/

Q17. Answer: B

Change alert compares the CURRENT window to a PRIOR window (e.g., last hour vs. last week). Anomaly compares to a continuous baseline. Threshold compares to a static number.

Doc: docs.datadoghq.com/monitors/types/log/?tab=threshold

Q18. Answer: B

Anomaly monitors learn a baseline (including daily/weekly seasonality) and alert when the value falls outside the expected band. Threshold monitors can't model trends; visual inspection is subjective; cardinality answers a different question.

Doc: docs.datadoghq.com/monitors/types/anomaly/

Q19. Answer: B

Logs-to-Metrics supports count AND distribution aggregations and up to 10 group-by tags per metric. It runs on ALL ingested logs (not just indexed), and the metrics are billed as custom metrics, not log volume.

Doc: docs.datadoghq.com/logs/log_configuration/logs_to_metrics/

Q20. Answer: B

View In Context is the 'show me what else was happening on this host/service around this moment' button — extremely useful when chasing root cause without full APM tracing. Trace-ID grouping is the separate 'View Trace' pivot.

Doc: docs.datadoghq.com/logs/explorer/

Q21. Answer: C

Export to Dashboard, Notebook, or Monitor are all available from the Explorer's Export menu. Saving as a Saved View is a SEPARATE action; the export menu doesn't auto-save the view.

Doc: docs.datadoghq.com/logs/explorer/

Q22. Answer: C

This is the textbook 'Logging Without Limits' pattern: keep ingestion (cheap), exclude from indexing (save money), generate metrics for dashboards/monitors, archive raw logs for the rare forensic deep-dive. Dropping at edge loses the forensic ability; full indexing is wasteful.

Doc: docs.datadoghq.com/logs/log_configuration/logs_to_metrics/

Domain 6 — Log Utilization

Monitors & Alerting · Metric Generation · Exporting

Domain 6 covers everything you DO with logs once they're in Datadog: monitors, custom metric generation, archives, rehydration, and forwarding. Expect at least one question on the log monitor query syntax (``logs("...").index("...").rollup("...").last("...") > N``) and several on cost-optimization patterns (the Logging Without Limits flow).

6.1 Log Monitors

- Standard log monitors run against INDEXED logs. Live monitors run against the Live Tail stream.
- Multi Alert: toggle to alert per group (per service, per host) — one monitor → many alerts.
- Notifications: ``@email``, ``@slack-channel``, ``@pagerduty-...``, ``@webhook-...``. Use template variables ``{{value}}``, ``{{threshold}}``, ``{{service.name}}``.
- Recovery threshold (lower than alert threshold) prevents flapping.
- Renotify keeps an unresolved alert visible by re-sending every X minutes.

Monitor type	Fires when	Use case
Threshold	Count over window crosses N	More than 50 errors per 5 min
Anomaly	Volume deviates from baseline	Unusual spike vs seasonal norm
Change alert	Volume changes vs prior window	Errors doubled vs last hour
Live	Any matching log in Live Tail	Independent of indexing, fires near-instantly

```
logs("status:error service:checkout")
  .index("main")
  .rollup("count")
  .last("5m") > 50
```

6.2 Generate metrics from logs

Datadog can derive CUSTOM METRICS from log volume or numeric values, ON INGEST, independent of indexing. This is the headline benefit of Logging Without Limits — your dashboards and monitors don't need the log to be indexed.

- Configuration: Logs → Generate Metrics.
- Query: which logs the metric counts.
- Aggregation: ``count`` (becomes a count metric) or ``distribution`` of a numeric attribute (sum/avg/percentiles preserved).
- Group-by: up to 10 tag dimensions become metric tags.
- Forward-only: metrics start emitting from the moment the rule is created.

Cardinality matters. Each unique tag-combo on a generated metric is a billable timeseries. Adding raw user IDs as a group-by tag explodes custom-metric cost. Restrict to low-cardinality dimensions.

6.3 Archives

Long-term cold storage in your own cloud bucket. Supported destinations: AWS S3, Azure Blob Storage, GCS. NOT supported: on-prem NFS.

- Archives contain ALL ingested logs that match the archive's filter — regardless of indexing.
- Stored as compressed JSON (`.json.gz`), one log per line.
- Hive-style partitioning: `dt=YYYY-MM-DD/hour=HH/...`.
- Encryption: SSE-S3, SSE-KMS, Azure native, GCS native (server-side); client-side encryption configurable.
- Retention is set on your bucket's lifecycle policy — Datadog imposes no maximum.

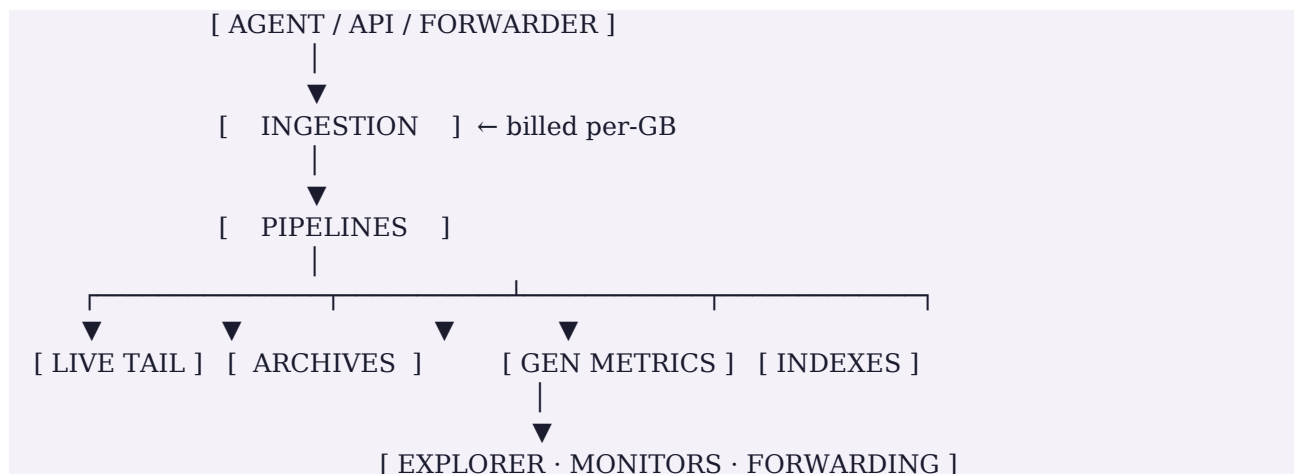
6.4 Rehydration

When you need to investigate logs older than the index retention, you rehydrate an archive: Logs → Configuration → Rehydrate from Archive. Pick the archive, time range, and optional query filter. Datadog re-reads, re-applies pipelines, and lands the result in a TEMPORARY index for the chosen retention. Counts toward your indexed-events bill but is bounded by your filters.

6.5 Forwarding

Datadog can forward a filtered subset of logs in real-time to external destinations: Splunk HEC, Elasticsearch / OpenSearch, generic HTTP webhooks, Microsoft Sentinel. Configure under Logs → Configuration → Forwarding. Useful when teams need logs in another SIEM but you want Datadog as the central pipeline.

6.6 Logging Without Limits in one diagram



6.7 Cost-control checklist

- Drop healthchecks at the Agent edge.
- Aggregate stack traces with `multi_line`.
- Generate metrics from high-volume access logs.
- Exclude high-volume, low-search-value logs from indexes.
- Use daily quotas on indexes for predictability.
- Right-size retention per index (7 days for debug, 30 for general, 90 for security).
- Archive everything cheaply; rehydrate when needed.

6.8 Exam takeaways

- Monitor query: `logs("<query>").index("<idx>").rollup("count").last("5m") > N``.
- Standard log monitors run on INDEXED logs; Live monitors on Live Tail.
- Generated metrics work even on EXCLUDED logs.
- Generated metrics: count + distribution; up to 10 group-by tags.
- Rehydration brings archived logs back into a temporary index.

- Forwarding pushes to Splunk / Elastic / HTTP / Sentinel.

Domain 6 — Quiz (22 hard questions)

Memorize the monitor query syntax and understand cardinality risk in logs-to-metrics.

Q1. Which log monitor syntax is correct?

- A. ``logs("service:checkout status:error").index("main").rollup("count").last("5m") > 50``
- B. ``logs.search("service:checkout status:error").rollup_count().last_5m > 50``
- C. ``SELECT count(*) FROM logs WHERE service='checkout' AND status='error' LAST 5m``
- D. ``monitor("service:checkout status:error", count, 5m, 50)``

Q2. Which destination is NOT supported for Datadog Log Archives?

- A. AWS S3
- B. Azure Blob Storage
- C. Google Cloud Storage (GCS)
- D. Local on-prem NFS share

Q3. Generated metrics from logs are billed as:

- A. Ingested logs (per GB).
- B. Custom metrics (per metric per host/per timestream).
- C. Index events.
- D. Free of charge.

Q4. Which is TRUE about log forwarding to third parties?

- A. Datadog forwards every log to all destinations by default.
- B. You can forward a filtered subset of logs to Splunk, Elasticsearch/OpenSearch, generic HTTP, or Microsoft Sentinel.
- C. Forwarding is configured via Saved Views.
- D. Forwarding requires re-ingestion of the logs.

Q5. A team archives all logs to S3 but forgot to enable rehydration. What happens?

- A. Archives are unreachable — they must be ingested via a separate Lambda.
- B. Rehydration is a one-time UI action; you simply configure it now and pick the archive + time window.
- C. Logs are permanently inaccessible.
- D. Datadog re-ingests automatically when you search them.

Q6. Which is FALSE about archives?

- A. They are stored compressed as JSON (`.json.gz``).
- B. They are partitioned in ``dt=YYYY-MM-DD/hour=HH/`` Hive-style layout.
- C. They contain only indexed logs.
- D. They support encryption (SSE-S3 / SSE-KMS / Azure native / GCS native).

Q7. A log monitor has the recovery threshold lower than the alert threshold. What does this configuration achieve?

- A. It prevents alerts from firing at all.
- B. It prevents flapping — once alerted, the value must drop below the recovery threshold to resolve.
- C. It reduces notification frequency.
- D. It enables anomaly mode.

Q8. True or False: A logs-to-metrics rule can retroactively create metrics from logs ingested before the rule was created.

- A. True
- B. False

Q9. Which template variable is INVALID in a Datadog log monitor message?

- A. `{{value}}`
- B. `{{threshold}}`
- C. `{{service.name}}`
- D. `{{log.body}}`

Q10. Why might you choose a `cardinality` rollup instead of `count` in a log monitor?

- A. To count distinct values of a facet (e.g., distinct users hit by errors), not total log volume.
- B. To improve query performance.
- C. To bypass index exclusion filters.
- D. Cardinality and count are interchangeable.

Q11. Which is TRUE about Live monitors (sometimes 'Logs Live monitor')?

- A. They search archived logs.
- B. They watch the Live Tail stream — fire near-instantly on any matching log, even if excluded from indexing.
- C. They require 10 minutes of warm-up.
- D. They only support `count` aggregations.

Q12. Which is FALSE about logs-to-metrics group-by tags?

- A. Up to 10 group-by tags per metric.
- B. Each unique combination of tag values becomes a new custom metric instance.
- C. High-cardinality tags (like raw user IDs) are safe to add — Datadog auto-aggregates.
- D. Tag values become metric tags.

Q13. Which is TRUE about log archives' retention?

- A. Datadog enforces a maximum of 90 days in archives.
- B. Retention is governed by your cloud provider's lifecycle policies (S3, Azure, GCS).
- C. Archives are retained for 15 months by default.
- D. Archives auto-delete when the source index expires.

Q14. A team's `nginx.requests`` generated metric should be tagged by ``service``, ``@http.method``, and ``@http.status_code``. Why might ``@http.url`` be a poor 11th tag?

- A. It exceeds the 10-tag limit.
- B. URLs are extremely high-cardinality — each unique URL becomes a new metric timeseries.
- C. Datadog doesn't support ``@*`` attributes as tags on generated metrics.
- D. Both A and B.

Q15. Which export option preserves the original log line text (not just an aggregation)?

- A. Export to dashboard widget.
- B. Logs-to-Metrics generation.
- C. Log Archives.
- D. Notebook (Analytics tab).

Q16. True or False: A single log monitor can target multiple indexes by using a wildcard ``index("*")``.

- A. True
- B. False

Q17. An archive's destination is configured but logs aren't appearing. Where is the most useful diagnostic?

- A. Datadog UI → Archives status page (shows last write time and any errors).
- B. Agent flare.
- C. Pipeline diagnostics page.
- D. The bucket's S3 access logs.

Q18. Which is TRUE about renotify settings on a log monitor?

- A. They notify only at recovery.
- B. They re-send the alert on a recurring interval if the issue is not resolved.
- C. They escalate to a different recipient.
- D. They are deprecated.

Q19. Which is FALSE about archive encryption?

- A. AWS S3 SSE-KMS is supported.
- B. AWS S3 SSE-S3 is supported.
- C. Client-side encryption is supported via configuration.
- D. Encryption is enabled by default with no possibility of disabling.

Q20. Which use case is BEST handled by Log Forwarding (vs. Archives)?

- A. Forensic storage of logs for 5+ years.
- B. Real-time mirroring of a subset of logs to a corporate SIEM that the security team prefers.
- C. Generating metrics from logs.
- D. Searching logs older than 90 days.

Q21. Generated metrics are most cost-effective when:

- A.** Group-by tags are limited to low-cardinality dimensions like ``service``, ``env``, ``region``, ``http_method``, ``status_code``.
- B.** Group-by tags include user IDs.
- C.** The log query is ``*``.
- D.** Each log generates one metric per attribute.

Q22. Which is the BEST description of ``Rehydrate from Archive``?

- A.** Datadog reads matching archive files, re-applies pipelines, and indexes them into a temporary index for a chosen retention window.
- B.** Datadog auto-recovers indexed logs that were accidentally excluded.
- C.** Archives are de-duplicated.
- D.** Logs are moved from archive to a cold-storage tier.

Domain 6 — Quiz (22 hard questions) — Answer Key & Explanations

Q1. Answer: A

Datadog log monitor query uses the `logs("<query>").index("<idx>").rollup("<agg>").last("<window>")` fluent syntax with a comparator. This is what the monitor editor shows in the source mode.`

Doc: docs.datadoghq.com/monitors/types/log/

Q2. Answer: D

Datadog Archives support S3, Azure Blob, and GCS. There's no first-class on-prem NFS option (you'd need Vector or another tool to mirror).

Doc: docs.datadoghq.com/logs/log_configuration/archives/

Q3. Answer: B

Logs-to-Metrics outputs are CUSTOM METRICS and count toward your custom-metric quota. The cost driver is the number of unique tag combinations on the metric (cardinality), so keep group-by tags lean.

Doc: docs.datadoghq.com/logs/log_configuration/logs_to_metrics/

Q4. Answer: B

Log Forwarding (Logs → Configuration → Forwarding) lets you push a subset (defined by a filter query) to external destinations like Splunk HEC, Elastic, generic HTTP, or Sentinel. Useful when teams need logs in another SIEM.

Doc: docs.datadoghq.com/logs/forwarding/

Q5. Answer: B

Rehydration is configured at the moment you need it — pick an archive, time range, optional query filter, and target retention. There's no 'forgot to enable' problem; it's an on-demand workflow.

Doc: docs.datadoghq.com/logs/log_configuration/rehydrating/

Q6. Answer: C

Archives contain ALL ingested logs that match the archive's filter, REGARDLESS of indexing. That's the whole point — you keep the long tail of logs cheaply outside the index.

Doc: docs.datadoghq.com/logs/log_configuration/archives/

Q7. Answer: B

A separate recovery threshold (below the alert) introduces hysteresis. The monitor alerts when the value crosses the alert threshold and only resolves when it falls below the recovery threshold. This prevents oscillation when the metric hovers around the alert level.

Doc: docs.datadoghq.com/monitors/configuration/?tab=thresholdalert

Q8. Answer: B

FALSE. Logs-to-Metrics is forward-only — it starts emitting metrics from the moment the rule is created. To analyze historical logs, you'd rehydrate them and aggregate in the Explorer.

Doc: docs.datadoghq.com/logs/log_configuration/logs_to_metrics/

Q9. Answer: D

`{{log.body}}` is not a standard template variable. Datadog provides variables for value, threshold, and tag-based group names (e.g., `{{service.name}}`, `{{host.name}}`). Multi-alert groupings expose their grouping tags as variables.

Doc: docs.datadoghq.com/monitors/notify/variables/?tab=is_alert

Q10. Answer: A

Cardinality counts DISTINCT values of a facet — useful when 'how many UNIQUE customers saw this error?' is more meaningful than 'how many error logs were emitted?' (a single retry storm can balloon count while cardinality stays small).

Doc: docs.datadoghq.com/monitors/types/log/

Q11. Answer: B

Live monitors are designed for the 'I want to know NOW if anything matches' use case, and they don't depend on indexing. They have a relatively limited query/agg set vs. standard monitors.

Doc: docs.datadoghq.com/monitors/types/log/

Q12. Answer: C

High-cardinality tags EXPLODE custom metric cost (each combo = one billable timeseries). Adding raw user IDs as group-by tags is a fast way to bankrupt your custom-metric budget. Restrict to low-cardinality dimensions.

Doc: docs.datadoghq.com/logs/log_configuration/logs_to_metrics/

Q13. Answer: B

Datadog writes to your S3/Azure/GCS bucket. Retention is your responsibility (via bucket lifecycle policies). You can keep archives forever or set tiered lifecycle to glacier-class storage. Datadog itself doesn't impose a retention limit.

Doc: docs.datadoghq.com/logs/log_configuration/archives/

Q14. Answer: D

Both: the 10-tag limit blocks adding an 11th, AND URLs are massively high-cardinality. Even within the 10-tag budget, raw URLs are a poor choice — bucket them first via Category processor or strip query strings.

Doc: docs.datadoghq.com/logs/log_configuration/logs_to_metrics/

Q15. Answer: C

Archives are the only path that preserves full raw log content. Dashboards/Notebooks see post-pipeline aggregates; metrics are derived counts.

Doc: docs.datadoghq.com/logs/log_configuration/archives/

Q16. Answer: A

TRUE. ``\*`` targets all indexes the user has access to. Useful when the same query should match in multiple indexes (e.g., when you've split prod and staging into separate indexes).

Doc: docs.datadoghq.com/monitors/types/log/

Q17. Answer: A

The Archives status page surfaces write errors (IAM, KMS, bucket policy mismatches) and last-success timestamps — the primary diagnostic surface for archive writes. Agent flare is irrelevant (archives are Datadog→cloud, not Agent-involved).

Doc: docs.datadoghq.com/logs/log_configuration/archives/

Q18. Answer: B

Renotify pushes the alert to recipients again every X minutes while unresolved. Useful to keep visibility on long-running incidents in busy channels. Escalation paths use service-level routing (PagerDuty etc.) not notify.

Doc: docs.datadoghq.com/monitors/configuration/?tab=thresholdalert#advanced-alert-conditions

Q19. Answer: D

Encryption is configurable — server-side options (SSE-S3, SSE-KMS, Azure native, GCS native) and client-side via additional setup. It is NOT mandatory by default in every config; you choose the encryption posture when setting up the archive destination.

Doc: docs.datadoghq.com/logs/log_configuration/archives/

Q20. Answer: B

Forwarding pushes logs in near-real-time to external systems — ideal for SIEM hand-offs. Archives are cold long-term storage. Metrics need Logs-to-Metrics. Old-log search uses rehydration.

Doc: docs.datadoghq.com/logs/forwarding/

Q21. Answer: A

Low-cardinality dimensions keep the metric's unique tag-combos manageable. Each combo is a billable timeseries; user IDs explode it. Tight log queries (not ``) also keep volume in check.

Doc: docs.datadoghq.com/logs/log_configuration/logs_to_metrics/

Q22. Answer: A

Rehydration re-reads the archive, re-applies pipelines, and lands the result in a temporary index for search. Counts toward indexed events for billing but is bounded by the time window and query filter you specify.

Doc: docs.datadoghq.com/logs/log_configuration/rehydrating/

Domain 7 — Log Troubleshooting

Agent Issues · Ingestion Issues

Domain 7 is the smallest scoring section but trips candidates because the questions are diagnostic scenarios. Learn the funnel: Agent healthy? Logs enabled? File readable? Pipeline parsing? Index inclusion? Exclusion filter? Facet present? Walk DOWN that list — never randomly poke.

7.1 The diagnostic funnel

1. Is the Agent running and healthy on the host?
2. Is log collection enabled (logs_enabled: true)?
3. Is the log file readable by the dd-agent user?
4. Does the integration config (conf.yaml) match the file path?
5. Are log_processing_rules silently dropping or scrubbing the log?
6. Are logs reaching Datadog (Live Tail)?
7. Are pipelines parsing as expected?
8. Are exclusion filters dropping the log from the index?
9. Are facets / standard attributes set up so search works?

7.2 Essential commands

```
sudo datadog-agent status          # All components, incl. Logs Agent block
sudo datadog-agent flare <ticket-id> # Bundle logs/config for Support
sudo datadog-agent stream-logs      # Live view of logs the Agent is processing
sudo -u dd-agent cat <path>         # Verify file readability
```

Kubernetes:

```
kubect exec -it <agent-pod> -n datadog -- agent status
kubect exec -it <agent-pod> -n datadog -- agent flare
```

Exam favorite. `datadog-agent flare <ticket-id>` is the canonical command to bundle Agent logs, configs, status, and runtime metrics for Datadog Support. It shows up on every Datadog cert exam.

7.3 Common symptoms → likely causes

Symptom	First check	Likely cause
No logs at all	<code>`agent status`</code>	<code>`logs_enabled: false`</code> , Agent stopped, network blocked
Live Tail OK, Explorer empty	Index inclusion/exclusion	100% exclusion, no inclusion match
Wrong timestamp	Pipelines → Date remapper	Missing or after a faulty grok
Wrong status	Status remapper	Mapping doesn't cover your <code>`level`</code> value
<code>`unknown`</code> source	Agent <code>`source:`</code>	Mismatched name vs OOTB pipeline (case-sensitive)
Stack traces split	Agent multi_line rule	Pattern doesn't match log start
<code>`permission denied`</code>	Linux ACLs	dd-agent lacks read on file OR traverse on parent
Logs delayed by minutes	Agent network	Buffer flush after reconnect
K8s logs missing	DaemonSet volumes / annotations	Missing <code>`/var/log/pods`</code> mount
Attribute not searchable	Facets	Custom JSON key not added as facet

7.4 Size limits to memorize

- Per log line: 1 MB (longer is truncated).
- Per HTTP intake payload: 5 MB.
- Logs-to-metrics group-by tags: 10.
- Index retention buckets: 3, 7, 15, 30, 60, 90 days.
- Live Tail window: ~15 minutes.

7.5 Datadog sites (region matters)

Region	`site:` value
US1	datadoghq.com
US3	us3.datadoghq.com
US5	us5.datadoghq.com
EU1	datadoghq.eu
AP1	ap1.datadoghq.com
US1-FED	ddog-gov.com

Trap. API keys are region-specific. An Agent on `site: datadoghq.eu` with a US1 API key fails silently from a user's perspective — `agent status` will show send errors. Always align `site:` and the API key.

7.6 Exam takeaways

- `datadog-agent status` is your first stop — read the Logs Agent block.
- `datadog-agent flare <ticket>` bundles everything for Support.
- `logs\_enabled: true` is THE master switch.
- Live Tail OK but Explorer empty → exclusion filter or wrong index.
- Wrong `site:` silently routes logs to the wrong region.
- File ACLs: dd-agent needs read on the file AND traverse on every directory above it.
- Multi-line patterns must match the START of a new log line.

Domain 7 — Quiz (18 hard questions)

Walk down the diagnostic funnel — every troubleshooting question maps to one of those nine steps.

Q1. A new K8s cluster's Agent DaemonSet is healthy (``agent status`` OK), but no logs appear in Datadog. ``logs_enabled`` and ``containerCollectAll`` are both true. The very first thing to check:

- A. Whether the Agent DaemonSet has volume mounts for ``/var/log/pods`` and ``/var/log/containers``.
- B. Whether the Pods have ``service`` annotations.
- C. Whether the index has too tight an inclusion filter.
- D. Whether the cluster has internet egress.

Q2. ``datadog-agent status`` shows ``LogsProcessed: 10000`` and ``LogsSent: 50``. The likely cause is:

- A. Network connectivity is dropping most packets.
- B. An ``exclude_at_match`` `log_processing_rule` is dropping most logs.
- C. The API key is invalid.
- D. The log files exceed 1 MB per line.

Q3. Which command bundles Agent logs, configs, status, and runtime metrics for Datadog Support?

- A. ``sudo datadog-agent flare <ticket-id>``
- B. ``sudo datadog-agent support-bundle``
- C. ``sudo datadog-agent dump``
- D. ``sudo datadog-agent export-logs``

Q4. Logs appear in Live Tail but NOT in the Log Explorer. Most likely cause:

- A. Index exclusion filter at 100%, or no index has an inclusion filter that matches.
- B. Wrong API key.
- C. Pipeline processors are dropping the logs.
- D. Agent has not been restarted.

Q5. An Agent in ``site: datadoghq.eu`` is using a US1 API key. What happens?

- A. Datadog auto-routes the logs to US1.
- B. The Agent's intake calls fail (HTTP 4xx) because the key is invalid for that region; logs are not delivered.
- C. Logs go to both regions.
- D. The Agent silently uploads to a generic fallback site.

Q6. An app emits 5 MB JSON logs per line. Datadog shows truncated entries. The limit is:

- A. 256 KB
- B. 1 MB
- C. 5 MB
- D. 10 MB

Q7. Multi-line Java stack traces are showing up as many separate log lines. The fix is a ``multi_line`` rule whose pattern matches:

- A. Lines containing ``at `` (stack frames).
- B. The start of a new log line, e.g., the leading timestamp or log level.
- C. The end of a stack trace.
- D. The class name in the exception.

Q8. A Linux app writes logs to ``/var/log/myapp/app.log`` owned by ``root:adm`` (mode 640). The Agent shows 'permission denied'. The MINIMAL fix is:

- A. Add the ``dd-agent`` user to the ``root`` group.
- B. Use ``setfacl`` to grant ``dd-agent`` read on the file AND traverse on the parent directory.
- C. Run the Agent as root.
- D. Set the file to 777.

Q9. Which command shows a real-time stream of logs as the Agent currently sees them (before transmission)?

- A. ``datadog-agent status``
- B. ``datadog-agent stream-logs``
- C. ``datadog-agent tail``
- D. ``datadog-agent logs --watch``

Q10. Logs are dramatically delayed (minutes behind real time). Which is the MOST LIKELY cause?

- A. Pipeline processor latency.
- B. The Agent's outbound network was disconnected and the on-disk buffer is now flushing.
- C. Index exclusion filter latency.
- D. Datadog backend throttling.

Q11. Logs have ``source: unknown`` and no parsing applies. The most likely cause:

- A. The Agent's ``source:`` field doesn't match any integration pipeline name (e.g., ``NGINX`` vs. ``nginx``, case-sensitive).
- B. The Agent is running an old version.
- C. The log line is malformed JSON.
- D. The pipeline is disabled.

Q12. What's the recommended way to debug a parser that's not working?

- A. Increase the Agent's log level to debug.
- B. Use the pipeline editor's 'sample log' preview — paste a real log and see which fields are extracted in real time.
- C. Rebuild the index.
- D. Open a support ticket.

Q13. Which is FALSE about Agent diagnostics?

- A. ``agent status`` is the first step.
- B. ``agent flare`` packages logs/config for Support.
- C. ``agent stream-logs`` shows the Agent's live log processing.
- D. ``agent ping`` validates connectivity to the intake.

Q14. Logs arrive but the timestamp is the ingest time, not the application time. Best fix:

- A. Reduce network latency.
- B. Add (or fix) a Date remapper consuming the extracted timestamp attribute, OR ensure the JSON ``timestamp`` field uses a recognized format.
- C. Increase Agent batch size.
- D. Enable Live Tail.

Q15. A pod's logs are being collected, but the ``service`` is wrong. Where do you correct it CLEANLY?

- A. Update ``ad.datadoghq.com/<container-name>.logs`` annotation on the Pod template (with the correct ``service``).
- B. Add a custom pipeline with a Service remapper.
- C. Edit the Agent DaemonSet config.
- D. Set a tag ``service:correct`` on the index.

Q16. Which Agent log file contains the most useful information for log-collection issues?

- A. ``/var/log/datadog/agent.log``
- B. ``/var/log/datadog/trace-agent.log``
- C. ``/var/log/datadog/process-agent.log``
- D. ``/var/log/datadog/system-probe.log``

Q17. A regex inside a ``multi_line`` pattern uses a positive lookahead. The Agent rejects the config. Why?

- A. Lookaheads are not allowed in YAML.
- B. The Agent uses Go's RE2 regex flavor, which doesn't support lookahead/lookbehind.
- C. The pattern is too long.
- D. Lookaheads conflict with the ``name`` field.

Q18. Which symptom is BEST diagnosed via the Pipelines page rather than the Agent?

- A. Wrong service set on a log.
- B. Logs not arriving at all.
- C. Permission denied on a host log file.
- D. Multi-line stack traces split.

Domain 7 — Quiz (18 hard questions) — Answer Key & Explanations

Q1. Answer: A

If the DaemonSet can't read the log files, no logs ever leave the cluster. Custom manifests frequently forget these mounts (the Helm chart sets them by default). After that, check egress / API key / site.

Doc: docs.datadoghq.com/containers/kubernetes/log/?tab=helm

Q2. Answer: B

``LogsProcessed - LogsSent`` is the count of logs the Agent intentionally dropped (via ``exclude_at_match`` or ``include_at_match`` that didn't match). A network problem would show up as send errors, not a Sent counter delta.

Doc: docs.datadoghq.com/agent/troubleshooting/

Q3. Answer: A

``datadog-agent flare <ticket>`` is the canonical command. It also uploads the bundle directly to Datadog Support if given a ticket ID. This question shows up on EVERY Datadog cert exam.

Doc: docs.datadoghq.com/agent/troubleshooting/send_a_flare/

Q4. Answer: A

Live Tail sees all ingested logs (post-pipeline). If they appear there but not in the Explorer, indexing has dropped them — either via exclusion filter or because they don't match any index's inclusion filter. Pipelines don't drop logs (they only mutate).

Doc: docs.datadoghq.com/logs/log_configuration/indexes/

Q5. Answer: B

Sites are isolated. EU1 intake rejects US1 keys. Symptoms: the Agent's ``agent status`` shows logs being processed but failing to send, with HTTP errors in the Agent log file. Fix: align the API key with the configured site.

Doc: docs.datadoghq.com/getting_started/site/

Q6. Answer: B

Individual log lines are capped at 1 MB; longer lines are truncated. (HTTP intake PAYLOAD is 5 MB, which is a separate limit on the batch size.) Reduce log size by trimming large fields like full request bodies or stack traces.

Doc: docs.datadoghq.com/api/latest/logs/#send-logs

Q7. Answer: B

``multi_line`` works by 'lines that do NOT match the pattern get appended to the previous line that did'. So the pattern must match the START of a new log line — typically a timestamp or ``INFO|ERROR`` prefix. Stack frames don't match → they get appended to the originating log line, exactly the desired outcome.

Doc: docs.datadoghq.com/agent/logs/advanced_log_collection/?tab=configurationfile#multi-line-aggregation

Q8. Answer: B

ACLs are the minimal-privilege fix — grant exactly what's needed without changing ownership or escalating the Agent. You need ``r`` on the file AND ``x`` on the parent path. Running as root or 777 are security regressions.

Doc: docs.datadoghq.com/agent/troubleshooting/permissions/

Q9. Answer: B

``datadog-agent stream-logs`` streams logs as the Agent processes them — invaluable for debugging ``log_processing_rules`` (you see exclusions/inclusions/masks live). The other commands are not valid for log streaming.

Doc: docs.datadoghq.com/agent/troubleshooting/debug_mode/

Q10. Answer: B

The Agent buffers to disk when it can't reach Datadog and replays the buffer on reconnect. Multi-minute delays after a network blip are normal until the buffer drains. Pipelines run in milliseconds and don't introduce minute-scale delay.

Doc: docs.datadoghq.com/agent/troubleshooting/

Q11. Answer: A

Integration pipelines trigger on EXACT, case-sensitive `source` values. `Nginx`, `NGINX`, `nginx-prod` would all miss the OOTB `nginx` pipeline. Either fix the `source` in the Agent config or build a custom pipeline for your value.

Doc: docs.datadoghq.com/logs/log_configuration/pipelines/?tab=source

Q12. Answer: B

The pipeline editor previews extraction live — fastest feedback loop for Grok and remapper iteration. Agent debug logging is for collection issues, not parsing.

Doc: docs.datadoghq.com/logs/log_configuration/pipelines/

Q13. Answer: D

There's no `agent ping` subcommand. To validate connectivity, look at the `Logs Agent` section of `agent status` (it reports the intake endpoint and bytes sent), or check Agent log files for HTTP errors.

Doc: docs.datadoghq.com/agent/configuration/agent-commands/

Q14. Answer: B

Datadog falls back to ingest time when no valid timestamp can be set. Either your Grok parser didn't produce a timestamp attribute, no Date remapper consumes it, or the JSON timestamp field isn't in a recognized format (RFC 3339 / ISO 8601 / common epoch variants).

Doc: docs.datadoghq.com/logs/log_configuration/processors/?tab=ui#date-remapper

Q15. Answer: A

Per-workload customization in K8s is done via annotations — that's the supported, declarative way. Service remappers work but feel like patching downstream. DaemonSet config is too coarse. Tags on indexes don't override the `service` attribute.

Doc: docs.datadoghq.com/containers/kubernetes/log/?tab=helm#autodiscovery

Q16. Answer: A

`agent.log` is the core Agent (including log collection) log. trace-agent is for APM; process-agent is for processes; system-probe is for NPM/network. For log-collection bugs, start with `agent.log` at debug verbosity.

Doc: docs.datadoghq.com/agent/troubleshooting/

Q17. Answer: B

Go's RE2 engine omits lookahead, lookbehind, and backreferences for performance/safety guarantees. Rewrite the pattern without them — most multi-line cases just need a simple timestamp/level prefix match.

Doc: docs.datadoghq.com/agent/logs/advanced_log_collection/

Q18. Answer: A

If the log IS arriving in Datadog but the `service` is wrong (e.g., a remapper is overwriting it), the Pipelines view shows which processors ran and the resulting attributes. 'Not arriving', 'permission denied', and 'multi-line split' all need Agent-side debugging.

Doc: docs.datadoghq.com/logs/log_configuration/pipelines/

Mock Exam 1 — mid-prep (Day 7)

25 questions, mixed domains. Target: 75 minutes (real exam pace). Take this after Week 1 (Domains 1-4 covered). Score by domain to spot weak areas before starting Week 2.

Mock Exam 1 — Questions

No notes, no docs. Answer all 25 then flip to the key.

Q1. A team's app emits unstructured logs and they want to extract a ``transaction_id`` field. Best processor:

- A. Lookup processor
- B. Grok parser
- C. Category processor
- D. Reference Table

Q2. A user runs ``service:checkout @http.status_code:[500 TO 599]``. They want the count of UNIQUE customer IDs hit by these errors over 15 min. The compute is:

- A. ``count``
- B. ``sum(@usr.id)``
- C. ``cardinality(@usr.id)``
- D. ``p95(@usr.id)``

Q3. Which is the master switch for log collection?

- A. ``logs.enabled`` in `datadog.yaml`
- B. ``logs_enabled: true`` in `datadog.yaml`
- C. ``logs: on`` in `conf.d/`
- D. ``DD_LOG_COLLECTION=true``

Q4. A team wants to suppress a noisy DEBUG line at the Agent edge (no ingestion bill). The right rule type:

- A. ``mask_sequences``
- B. ``include_at_match``
- C. ``exclude_at_match``
- D. ``multi_line``

Q5. Which fact about reserved attributes is TRUE?

- A. Reserved attributes use the ``@`` prefix.
- B. There are seven: host, service, source, status, message, timestamp, tags.
- C. They are configurable per organization.
- D. They include ``@http.method``.

Q6. A team needs to retain logs for 30 days for security audits at minimum cost. The BEST architecture is:

- A. Index everything for 30 days.
- B. Index for 7 days; archive everything; rehydrate for security investigations.
- C. Send all logs to a third-party SIEM.

D. Drop all logs at the Agent.

Q7. An index has exclusion filter ``source:nginx @http.status_code:200`` at 90%. What is excluded?

- A. All NGINX 200s.
- B. 90% of NGINX 200s (sampled).
- C. 10% of NGINX 200s.
- D. Nothing — filters can't sample.

Q8. Which is the BEST query to find logs from any web-* host with status 5xx OR 4xx in prod?

- A. ``env:prod host:web-* @http.status_code:[400 TO 599]``
- B. ``env:prod AND host:web* AND status:5xx OR 4xx``
- C. ``@host:web-* env:prod (status:5xx OR status:4xx)``
- D. ``env:prod host:web-* @http.status_code:>=400``

Q9. A pipeline contains processors in this order: Grok parser → Date remapper. Status is correct but timestamp is wrong. The most likely cause:

- A. Pipeline filter doesn't match.
- B. Grok didn't capture the timestamp attribute that Date remapper expects.
- C. Date remapper is BEFORE Grok.
- D. Status is unrelated to timestamp.

Q10. Live Tail's coverage window is:

- A. 1 minute
- B. 5 minutes
- C. ~15 minutes
- D. 1 hour

Q11. Which visualization supports the most group-by dimensions?

- A. Top List
- B. Pie Chart
- C. Geomap
- D. Table

Q12. An incoming JSON log has key ``severity: error``. Datadog will:

- A. Ignore it; only ``level`` is auto-detected.
- B. Auto-promote it to ``status:error`` without a Status remapper.
- C. Treat it as a custom ``@severity`` attribute.
- D. Reject the log as malformed.

Q13. Which file does ``datadog-agent flare`` send to Datadog Support?

- A. Just the most recent ``agent.log`` line.

- B. A zipped bundle of agent logs, configs, status, and runtime info.
- C. A list of Kubernetes pods.
- D. The API key.

Q14. A team enables `container_collect_all: true`. The Agent reads logs from:

- A. Files inside each container's filesystem.
- B. Each container's STDOUT/STDERR (via the runtime's log files under `/var/log/pods/`).
- C. Docker inspect API.
- D. Custom log socket per container.

Q15. Which is FALSE about log archives?

- A. Supports AWS S3, Azure Blob, and GCS.
- B. Stored as compressed JSON with one log per line.
- C. Excluded logs ARE archived if the archive matches.
- D. Archived logs are searchable directly in the Log Explorer.

Q16. A team needs to alert when distinct customer count seeing errors spikes vs. baseline. The right monitor type is:

- A. Threshold
- B. Anomaly
- C. Change alert
- D. Composite

Q17. Which is TRUE about the Sensitive Data Scanner?

- A. Runs on the Agent before transmission.
- B. Runs server-side after ingestion, before indexing AND archive — so redactions persist in archives.
- C. Only redacts; cannot hash.
- D. Custom regex is not supported.

Q18. Two indexes: `prod-errors` (status:error env:prod`)` and `prod-all` (env:prod`)`. A log `service:foo env:prod status:error`` goes to:

- A. Both indexes.
- B. `prod-errors`` only (first matching index wins).
- C. `prod-all`` only.
- D. Whichever has more retention.

Q19. An app sends 1.5 MB JSON log lines. What happens?

- A. Logs are accepted as-is.
- B. Logs are TRUNCATED to 1 MB (the per-line cap).
- C. Logs are rejected.
- D. Datadog auto-compresses them to fit.

Q20. True or False: The Patterns view can be used to one-click create an exclusion filter for a noisy pattern.

- A. True
- B. False

Q21. Which compute is invalid in Datadog log Analytics?

- A. count
- B. cardinality
- C. median
- D. distinct

Q22. True or False: A facet must be created BEFORE logs are ingested for that attribute to be searchable.

- A. True
- B. False

Q23. Which is TRUE about `View In Context`?

- A. It shows the APM trace tied to the log.
- B. It opens a stream of logs from the same host AND service around the log's timestamp.
- C. It re-executes the pipeline.
- D. It exports the log to a Notebook.

Q24. Which is FALSE about logs-to-metrics?

- A. Generates metrics from ALL ingested logs (independent of indexing).
- B. Supports count and distribution aggregations.
- C. Supports up to 10 group-by tags.
- D. Generated metrics are retained for the same period as the source index.

Q25. An incoming JSON log has `timestamp: "2026-05-15T10:00:00Z"`. Datadog will:

- A. Use it as the official timestamp automatically (RFC 3339 detected).
- B. Ignore it (only `@timestamp` is auto-detected).
- C. Drop the log.
- D. Always defer to ingest time.

Mock Exam 1 — Questions — Answer Key & Explanations

Q1. Answer: B

Grok extracts structured fields from unstructured text via named regex patterns. Lookup translates; Category buckets; Reference Table is for shared lookup CSVs.

Doc: docs.datadoghq.com/logs/log_configuration/parsing/

Q2. Answer: C

Distinct count of `@usr.id` = cardinality. Sum/p95 need numeric measures; count returns total log volume not unique users.

Doc: docs.datadoghq.com/logs/explorer/analytics/

Q3. Answer: B

The exact key is `logs_enabled: true` at the top level of `datadog.yaml` (env-var equivalent: `DD_LOGS_ENABLED=true`).

Doc: docs.datadoghq.com/agent/logs/

Q4. Answer: C

`exclude_at_match` drops lines matching the regex at the Agent. Edge-level drop avoids the ingestion fee entirely.

Doc: docs.datadoghq.com/agent/logs/advanced_log_collection/

Q5. Answer: B

Reserved attributes are the seven top-level fields WITHOUT `@`. Standard attributes (with `@`) like `@http.method` are different.

Doc: docs.datadoghq.com/logs/log_configuration/attributes_naming_convention/

Q6. Answer: B

Pair short index retention for routine search with long archive retention (cheap S3) for compliance. Rehydrate on demand.

Doc: docs.datadoghq.com/logs/log_configuration/archives/

Q7. Answer: B

Exclusion percentage is the DROP rate. 90% means 9 of every 10 matching logs are dropped; 1 is kept.

Doc: docs.datadoghq.com/logs/log_configuration/indexes/#exclusion-filters

Q8. Answer: A

Range query is the cleanest way to capture all 4xx+5xx. `host:web-*` uses wildcard correctly. The other options have syntax issues: lowercase boolean, `@host`, `>=` not supported in numeric search.

Doc: docs.datadoghq.com/logs/explorer/search_syntax/

Q9. Answer: B

If status is correct, the pipeline IS matching. Date remapper failing while status works means Grok extracted `@level` but not the timestamp field. Inspect the Grok preview.

Doc: docs.datadoghq.com/logs/log_configuration/processors/?tab=ui#date-remapper

Q10. Answer: C

Live Tail is a streaming view with approximately 15 minutes of buffer.

Doc: docs.datadoghq.com/logs/explorer/live_tail/

Q11. Answer: D

Table, Timeseries, Tree Map, Pie Chart, and Geomap each support up to 4 group-by dimensions. Top List supports only 1.

Doc: docs.datadoghq.com/logs/explorer/analytics/

Q12. Answer: B

Both `level` and `severity` JSON keys auto-promote to the reserved `status` attribute, with common case values translated.

Doc: docs.datadoghq.com/logs/log_configuration/attributes_naming_convention/

Q13. Answer: B

`flare` packages diagnostic info (logs, configs, status, runtime metrics). It scrubs the API key by default.

Doc: docs.datadoghq.com/agent/troubleshooting/send_a_flare/

Q14. Answer: B

Container log collection reads STDOUT/STDERR captured by the runtime, not files inside the container.

Doc: docs.datadoghq.com/containers/kubernetes/log/

Q15. Answer: D

Archives are cold storage and are NOT directly searchable. To search them, rehydrate into a temporary index. The other statements are true.

Doc: docs.datadoghq.com/logs/log_configuration/archives/

Q16. Answer: B

Anomaly tracks deviation from a learned baseline (with seasonality). Threshold uses a fixed value; Change compares two windows; Composite chains other monitors.

Doc: docs.datadoghq.com/monitors/types/anomaly/

Q17. Answer: B

Sensitive Data Scanner is server-side and runs before indexing AND archive write. It supports redact, hash, partial-redact, and tag-only actions, and custom regex rules.

Doc: docs.datadoghq.com/sensitive_data_scanner/

Q18. Answer: B

Indexes evaluate in order; a log goes to the FIRST matching index only. Pipelines, in contrast, run for every match.

Doc: docs.datadoghq.com/logs/log_configuration/indexes/

Q19. Answer: B

Per-line cap is 1 MB; longer lines are truncated. The HTTP payload cap is a separate 5 MB limit on the batch.

Doc: docs.datadoghq.com/api/latest/logs/#send-logs

Q20. Answer: A

TRUE. From the Patterns view, you can convert any pattern directly into an exclusion filter (or drop processor in a pipeline) for the affected query.

Doc: docs.datadoghq.com/logs/explorer/patterns/

Q21. Answer: D

Use `cardinality` for distinct counts. `distinct` is not a Datadog log aggregator name. Median = p50.

Doc: docs.datadoghq.com/logs/explorer/analytics/

Q22. Answer: B

FALSE. Facets created later apply retroactively — Datadog re-indexes on the fly. `@key:value` text search works even without faceting.

Doc: docs.datadoghq.com/logs/explorer/facets/

Q23. Answer: B

View In Context surfaces nearby logs from the same host and service — your fastest way to see what was happening around the moment of an error.

Doc: docs.datadoghq.com/logs/explorer/

Q24. Answer: D

Custom metrics have their own retention (typically 15 months by default). They are not tied to source-index retention.

Doc: docs.datadoghq.com/logs/log_configuration/logs_to_metrics/

Q25. Answer: A

Datadog auto-detects `timestamp` (RFC 3339 / ISO 8601 and common variants) and uses it as the official log timestamp without requiring a Date remapper.

Doc: docs.datadoghq.com/logs/log_configuration/attributes_naming_convention/

Mock Exam 2 — final dress rehearsal (Day 13)

25 questions, harder scenarios and edge cases. Target: 80%+. If you score below 75%, spend Day 14 reviewing your weakest domain before the exam.

Mock Exam 2 — Questions

This mock is calibrated harder than the real exam. Survive this and you're ready.

Q1. Your CFO wants a 40% reduction in log spend. Current pattern: index all NGINX access logs at 100%, 30-day retention; never search older than 7 days; dashboards built on log counts. What's the highest-leverage single change?

- A. Drop ALL NGINX logs at the Agent.
- B. Reduce retention to 7 days, exclude access logs from index at 100%, generate `nginx.requests` metric for dashboards, archive raw logs.
- C. Move NGINX to a separate index with the same retention.
- D. Forward NGINX logs to S3 from NGINX directly, bypassing Datadog.

Q2. A multi-region setup: US-East cluster + EU-West cluster, one Datadog org (US1). Both Agents are configured `site: datadoghq.com`. EU GDPR officer demands EU-customer logs never leave the EU. The correct architecture is:

- A. Add a Sensitive Data Scanner rule that masks EU-customer fields.
- B. Provision a separate Datadog org in EU1 (datadoghq.eu) for EU Agents; logs stay in EU intake/storage. Cross-org dashboards live in either side.
- C. Use exclusion filters to remove EU-customer logs from the index.
- D. Forward EU logs from US1 back to an EU S3 bucket.

Q3. An engineer added the field `request\_status` to their app's JSON logs. The OOTB HTTP dashboard still shows zero traffic. Why?

- A. Faceting needs to be enabled.
- B. OOTB HTTP dashboards rely on `@http.status\_code`, not `@request\_status`. Add a Standard Attribute remapper.
- C. The Agent doesn't parse JSON.
- D. The dashboard is read-only.

Q4. A team has 3 pipelines: P1 (filter `\*`), P2 (`source:java`), P3 (`service:billing`). A log with `source:java service:billing` triggers:

- A. Only P1 (first matching).
- B. All three pipelines, in order P1 → P2 → P3.
- C. P2 and P3, but not P1.
- D. Only P3 (most specific).

Q5. A Kubernetes deployment uses an init container that emits valuable logs once at startup. The main container also emits logs. Both go to the Agent — but the init container's logs never appear in Datadog. Why?

- A. Init containers cannot be log-collected.
- B. Init containers exit before the Agent's polling can pick them up if the pod is short-lived; pod annotations should include the init container name, and ensure log buffer captures startup.
- C. The Agent only reads the main container's logs by design.
- D. Init container logs are sent to a different region.

Q6. Which pipeline filter would match logs from ``service` `checkout-v1` OR `checkout-v2` in `env:prod``?

- A. ``service:checkout-* env:prod``
- B. ``service:(checkout-v1 OR checkout-v2) AND env:prod``
- C. Either of the above.
- D. ``service:checkout AND version:(v1 OR v2) AND env:prod``

Q7. A logs-to-metrics rule with group-by ``service`, `env`, `region`, `@http.method`, `@http.status_code`` is created. The team now wants to add ``@usr.id``. The risk is:

- A. It exceeds the 10-tag limit.
- B. User IDs are extremely high cardinality — each unique combo becomes a new billable custom-metric timeseries; costs explode.
- C. User IDs can't be used as metric tags at all.
- D. Pipelines would need to be reconfigured.

Q8. A team's app emits both INFO and ERROR JSON logs. They want INFO indexed for 7 days and ERROR for 90 days. The architecture:

- A. Set 90-day retention on the main index.
- B. Two indexes: ``default-info`` (``status:info``, retention 7d) and ``errors`` (``status:(error OR warning OR critical OR alert OR emergency)``, retention 90d). Order them so the errors index is evaluated FIRST.
- C. Set per-log retention via a pipeline tag.
- D. Use exclusion filters to drop INFO after 7 days.

Q9. Multi-line aggregation rule with pattern ``\d{4}-\d{2}-\d{2}``. A new Spring Boot service emits stack traces. Some still get split. The likely cause:

- A. The log start contains a colored ANSI prefix (e.g., ``[INFO]``) before the date — the pattern doesn't match the literal start.
- B. Multi-line patterns only work for Python.
- C. The Agent doesn't support Spring Boot.
- D. The pattern needs lookahead.

Q10. An app uses a custom trace correlation field ``@app.requestId`` matching ``dd.trace_id``. Logs aren't appearing in the Trace side-panel. Best fix:

- A. Add a Trace ID remapper that promotes ``@app.requestId`` to ``dd.trace_id``.
- B. Manually re-emit logs with ``dd.trace_id``.

- C. Add a Service remapper.
- D. Disable APM.

Q11. A team's `agent status` shows 'permission denied' for `/var/log/foo/bar.log`. They `chmod 644` the file but the Agent still can't read it. The likely reason:

- A. The Agent caches permissions for 1 hour.
- B. The parent directory `/var/log/foo/` lacks execute (search) permission for `dd-agent` — POSIX requires `x` on every directory in the path.
- C. The Agent is running as `root` so chmod doesn't matter.
- D. The file is held open by another process.

Q12. A SOC wants to alert when ANY log line contains a JWT token leak in any service. The MOST EFFICIENT approach:

- A. A standard log monitor with regex on `message`.
- B. A Sensitive Data Scanner rule that tags matching logs (e.g., `sensitive\_data:jwt`), then a monitor on `sensitive\_data:jwt`.
- C. Manual sampling in Live Tail.
- D. An Agent `mask\_sequences` rule plus a monitor.

Q13. A new pipeline produces correctly parsed logs in the preview but production logs still show `unparsed`. The most likely cause:

- A. The pipeline filter doesn't match the production log's `source`/`service`/`env`.
- B. Datadog needs 24 hours to propagate pipelines.
- C. The pipeline is in a draft state.
- D. Production logs are encrypted.

Q14. Which Watchdog Insights capability is FALSE?

- A. Automatically detects unusual log patterns.
- B. Requires creation of detection rules to function.
- C. Available from the Logs landing page.
- D. Runs continuously without configuration.

Q15. A team needs to forward `audit\_event:login\_failure` events to Splunk while keeping everything else in Datadog. Best setup:

- A. Re-emit logs from the app to both destinations.
- B. Datadog Log Forwarding with destination Splunk HEC + filter query `audit\_event:login\_failure`.
- C. Send all logs to S3 and have Splunk pull from S3.
- D. Use Vector to fan out logs.

Q16. A user creates an index with a daily quota of 5M events. Logs at the quota boundary:

- A. Are dropped from ingestion.

- B. Continue to be ingested and archived, but stop being indexed for the rest of the day.
- C. Are buffered until the next day.
- D. Are silently moved to another index.

Q17. Which of the following is NOT a valid Datadog log destination via Log Forwarding?

- A. Splunk HEC
- B. Elasticsearch / OpenSearch
- C. Microsoft Sentinel
- D. PostgreSQL database

Q18. A team's `@latency_ms` measure shows correct values in Top List but the timeseries shows zero. Likely cause:

- A. Timeseries doesn't support measures.
- B. The selected compute is `count` (which counts log volume, not the measure) — change to `avg` or `p95` of `@latency_ms`.
- C. Timeseries requires 4 group-by dimensions.
- D. The measure unit isn't set.

Q19. Which is TRUE about the Live (Live Tail) monitor type?

- A. Watches indexed logs only.
- B. Watches the Live Tail stream — fires near-instantly even for logs not in any index.
- C. Requires 10 minutes of warm-up.
- D. Available only on Enterprise plans.

Q20. Pipeline processors execute:

- A. In parallel for performance.
- B. In their declared order, top to bottom.
- C. In alphabetical order.
- D. In random order if not specified.

Q21. A team's pipelines are correct in dev but parse fails in prod. The dev and prod logs differ in:

- A. Pipeline filters are env-specific.
- B. The log format itself (dev uses `logfmt`, prod uses raw text); the same Grok pattern can't match both.
- C. The Agent version.
- D. The Datadog site.

Q22. Which is the BEST description of a Datadog `Saved View` URL parameter?

- A. `?id=<view_id>`
- B. `?saved_view=<view_id>` plus optional `&query=...` overrides.
- C. `?view=<name>`
- D. `?vid=<view_id>`

Q23. An Agent on a Windows VM tails `C:\logs\app.log`. After a UTF-16 encoding mismatch, logs appear garbled. The fix:

- A. Convert the file to UTF-8 outside Datadog.
- B. Set ``encoding: utf-16-le`` on the log entry in the Agent config.
- C. Switch to ``type: windows_event``.
- D. There is no fix — Datadog only supports UTF-8.

Q24. Which is FALSE about log ingestion costs?

- A. Ingestion is billed per GB ingested.
- B. Edge ``exclude_at_match`` reduces ingestion cost.
- C. Index exclusion filters reduce ingestion cost.
- D. Generated metrics are billed separately from ingestion.

Q25. A team has logs landing with ``source: java-app`` but no integration pipeline matches. The fix:

- A. Datadog will retroactively parse them in 24h.
- B. Either change ``source`` to ``java`` to trigger the OOTB Java pipeline, OR build a custom pipeline with filter ``source:java-app``.
- C. Set ``service:java-app`` instead.
- D. Move to a different region.

Mock Exam 2 — Questions — Answer Key & Explanations

Q1. Answer: B

Combines retention right-sizing, exclusion (no index cost), metric generation (cheap dashboards), and archival (cheap forensics) — the textbook `Logging Without Limits` flow. Dropping at Agent kills forensic ability; new index at same retention doesn't save much; bypassing Datadog loses the integration.

Doc: docs.datadoghq.com/logs/log_configuration/logs_to_metrics/

Q2. Answer: B

Data residency requires the data to never enter the wrong region's intake — which only happens by using a region-specific org with EU-resident Agents pointing to `datadoghq.eu`. Masking, exclusion, or downstream forwarding all happen AFTER the data has reached US intake.

Doc: docs.datadoghq.com/data_security/logs/

Q3. Answer: B

Standard attribute remappers map custom field names into the Datadog standard namespace so OOTB dashboards and analytics work. Faceting alone won't help OOTB dashboards.

Doc: docs.datadoghq.com/logs/log_configuration/attributes_naming_convention/

Q4. Answer: B

Pipelines do not short-circuit. Every matching pipeline runs in display order. This is why filter narrowness matters: P1 with `\*\*` wastes CPU on every log.

Doc: docs.datadoghq.com/logs/log_configuration/pipelines/

Q5. Answer: B

Init containers ARE supported, but timing matters: pod annotations must reference the init container name (`ad.datadoghq.com/<init-container-name>.logs`), and the Agent needs to see the log files before the pod is deleted. Increase `tailing\_threshold` or use `start\_position: beginning` if needed.

Doc: docs.datadoghq.com/containers/kubernetes/log/

Q6. Answer: C

Both `service:checkout-\*` (wildcard) and `service:(checkout-v1 OR checkout-v2)` (explicit OR) work. The latter is safer when other service names like `checkout-internal` exist that you don't want to include.

Doc: docs.datadoghq.com/logs/explorer/search_syntax/

Q7. Answer: B

User IDs (and other unique-per-request identifiers) explode metric cardinality. Even within the 10-tag budget, picking the wrong tag dimension is the #1 way teams blow up their custom-metric bill.

Doc: docs.datadoghq.com/logs/log_configuration/logs_to_metrics/

Q8. Answer: B

Per-log retention is achieved by routing logs into different indexes with different retention. Inclusion filter evaluation is in display ORDER, so put the most specific (errors) first to capture them; INFO falls through to the broader index.

Doc: docs.datadoghq.com/logs/log_configuration/indexes/

Q9. Answer: A

Multi-line patterns match the START of a line. If your logs have an ANSI color escape, timestamp suffix, or any prefix before the date, the pattern misses. Adjust to allow optional leading characters: e.g., `[?w+]?s\*\d{4}-\d{2}-\d{2}` or anchor with the actual prefix.

Doc: docs.datadoghq.com/agent/logs/advanced_log_collection/

Q10. Answer: A

The Trace ID remapper is exactly for this case — it tells Datadog 'use this attribute as the canonical trace ID'. After applying, logs link to traces in the Trace side-panel.

Doc: docs.datadoghq.com/logs/log_configuration/processors/?tab=ui#trace-id-remapper

Q11. Answer: B

POSIX needs read on the file AND execute (traverse) on each directory in the path. `chmod 644 file` doesn't help if `/var/log/foo/` is mode 700 root:root. Fix with ACLs or chmod o+x on the directory.

Doc: docs.datadoghq.com/agent/troubleshooting/permissions/

Q12. Answer: B

Sensitive Data Scanner adds a `sensitive\_data` tag to scanned logs that matched. A monitor on that tag catches every leak across all services. Agent rules don't tag; manual sampling isn't a system.

Doc: docs.datadoghq.com/sensitive_data_scanner/

Q13. Answer: A

If preview works but prod logs aren't parsed, the issue is the FILTER, not the processors. Pipelines propagate within seconds; there's no draft state once saved.

Doc: docs.datadoghq.com/logs/log_configuration/pipelines/

Q14. Answer: B

Watchdog runs WITHOUT user-created detection rules. Those belong to Cloud SIEM. Watchdog is the auto-anomaly layer for logs (and metrics, traces).

Doc: docs.datadoghq.com/watchdog/

Q15. Answer: B

Datadog Log Forwarding to Splunk HEC is built for this. Filter the forwarded subset using the regular query syntax. Vector also works but adds operational overhead when forwarding is built-in.

Doc: docs.datadoghq.com/logs/forwarding/

Q16. Answer: B

The daily quota is a hard ceiling on indexed events. Excess logs flow through ingestion, pipelines, archives, and metric generation, but bypass the index. Resets at the day boundary.

Doc: docs.datadoghq.com/logs/log_configuration/indexes/#index-rate-limits

Q17. Answer: D

Supported forwarding destinations are Splunk HEC, Elastic/OpenSearch, Microsoft Sentinel, and generic HTTP webhooks. Direct DB writes are not supported.

Doc: docs.datadoghq.com/logs/forwarding/

Q18. Answer: B

`count` counts logs. To plot the actual values of `@latency\_ms`, choose an aggregation of the measure (avg, sum, p95). All visualizations support measures and counts equally.

Doc: docs.datadoghq.com/logs/explorer/analytics/

Q19. Answer: B

Live monitors evaluate the Live Tail stream — independent of indexing. The fastest way to fire on the appearance of any matching log.

Doc: docs.datadoghq.com/monitors/types/log/

Q20. Answer: B

Always declared order. Reorder when extractions need to happen before remappers consume them (the canonical example: Grok must precede Date remapper that consumes the extracted timestamp).

Doc: docs.datadoghq.com/logs/log_configuration/processors/

Q21. Answer: B

Parse failure across environments usually traces to LOG FORMAT divergence. Standardize on JSON across environments, or write a pipeline per format.

Doc: docs.datadoghq.com/logs/log_configuration/parsing/

Q22. Answer: B

The canonical URL parameter is `?saved\_view=<id>`. Combine with `&query=...` to apply ad-hoc filters on top of the saved view.

Doc: docs.datadoghq.com/logs/explorer/saved_views/

Q23. Answer: B

The Agent's log entry has an `encoding:` field for non-UTF-8 files (common on Windows where logs are UTF-16 LE). Setting this lets the Agent decode correctly before sending.

Doc: docs.datadoghq.com/agent/logs/?tab=tailfiles

Q24. Answer: C

Index exclusion filters reduce INDEX cost but the log has already been ingested (billed). Only Agent-edge drop saves ingestion. Generated metrics count toward custom metric quota, separate from log billing.

Doc: docs.datadoghq.com/logs/log_configuration/

Q25. Answer: B

Integration pipelines trigger on exact `source` matches. Custom `source` values won't pick up OOTB parsing — either align with the documented `source` name or build a custom pipeline.

Doc: docs.datadoghq.com/logs/log_configuration/pipelines/?tab=source

Quick reference cheat sheet

Print this. One page of the things that show up on every practice run and in most real exam questions.

Topic	What to remember
Reserved attributes (no @)	host · service · source · status · message · timestamp · tags
Status values	emergency · alert · critical · error · warning · notice · info · debug · OK
Standard attribute namespaces	@http.* · @network.* · @error.* · @usr.* · @db.* · @duration · @trace_id / @span_id
Master switches	`logs_enabled: true` (datadog.yaml) · `DD_LOGS_ENABLED=true` (env) · `container_collect_all: true` (Docker/K8s)
log_processing_rules types	exclude_at_match · include_at_match · mask_sequences · multi_line
Pipeline processor order	Grok → Date remapper → Status remapper → Service/Message remappers → Std attr remappers → Enrich (GeoIP/UA/Category/Lookup) → Restructure
Search — tags & reserved	no @ → service:checkout env:prod host:web-01 status:error source:nginx
Search — attributes	@ prefix → @http.status_code:500 @duration:>1000000000
Range queries	[a TO b] inclusive · {a TO b} exclusive · * is open bound
Boolean operators	UPPERCASE only: AND OR NOT · `~` shortcut for NOT
Wildcards	Trailing/mid: OK (service:payment-*) · Leading slow (*error)
Existence	@usr.id:* exists · -@usr.id:* missing
Live Tail vs Explorer	Live: 15-min, ALL ingested · Explorer: indexed only, retention 3-90d
Visualization group-by limits	Top List: 1 · Timeseries / Table / TreeMap / Pie / Geomap: 4
Computes	count · cardinality(facet) · sum/avg/min/max(measure) · pcXX(measure)
Monitor query	logs("query").index("idx").rollup("count").last("5m") > N
Sizing	Log line 1 MB · HTTP payload 5 MB · Logs-to-Metrics 10 group-by tags · Retention 3/7/15/30/60/90 d
Diagnostic commands	datadog-agent status · datadog-agent flare <ticket> · datadog-agent stream-logs
Sites	US1 datadoghq.com · US3 us3.datadoghq.com · US5 us5.datadoghq.com · EU1 datadoghq.eu · AP1 ap1.datadoghq.com
Symptom → first place	No logs → agent status · Live Tail OK + Explorer empty → exclusion filter / index routing · Wrong timestamp → Date remapper / Grok · `unknown` source → source spelling case-sensitive

Exam-day strategy & final tips

- Read every question carefully — Datadog distractors are designed to be plausible. Look for absolute words (always, never, only) which often signal wrong answers.
- If two answers seem correct, pick the one that aligns with Datadog's STRONGEST recommendation. Emit JSON, set unified service tags, drop at the edge — these are repeatedly the right choices.
- When stuck, eliminate. Knock out two obviously wrong distractors, then pick the more specific of the remaining two.
- Use Live Tail when the question scenario is `verify that logs are arriving for a new service` — even if the index isn't mentioned. Almost always the intended answer.
- Exclusion filter $\neq$ ingestion drop. This pattern shows up in multiple disguises — burn it into memory.
- If a question mentions K8s log collection and something isn't working, default-check volume mounts (`/var/log/pods`) and annotation container-name match — they trip everyone.
- Manage time: 90 questions in 120 minutes = 80 seconds each. Mark-and-skip if you're stuck after 90 seconds; return at the end.
- Flag and review: Kryterion lets you flag questions for review. Use it liberally early in the exam; come back with fresh eyes.
- Stay hydrated and take a small bathroom break only if it really helps — the clock keeps running.
- After you finish: you get the score immediately. Pass — celebrate. Fail — note your weakest domains, regroup, and retake (up to 3 attempts per 180-day window).

You've got this

Two weeks at 1.5-2 hours per day, anchored by hands-on Datadog work and roughly 200 hard practice questions, is more than enough preparation if you commit to it. The single biggest predictor of passing is taking the quizzes seriously — not skimming the answer key, but pausing on each miss and re-reading the relevant section until the concept is solid.

When you sit the exam, scan the cheat sheet that morning, drink water, sit early afternoon if you can choose, and trust the prep. Boa sorte!